UNIVERSITÀ
DI SIENA
1240

University of Siena

Department of Information Engineering and Mathematics

PathNet: A* Search for Multilayer Perceptron Training

Kevin Gagliano

Jacopo Andreucci

Supervisor:

Prof. Edmondo trentin

Academic Year 2024-2025

Contents

Chapter 1

The A* Search Algorithm

1.1 Overview and Context

The **A* (A-star) search algorithm** is a cornerstone of Artificial Intelligence and computer science, highly effective for pathfinding and optimal graph traversal. Developed in 1968 by Hart, Nilsson, and Raphael as an extension of Dijkstra's algorithm, A* is classified as an *informed search* or *best-first search* method. Unlike uninformed searches, A* strategically explores the solution space by using estimated knowledge about the goal, enabling it to efficiently find the shortest path between a start and goal node. Its classical applications include video game pathfinding, network routing, and logistical planning.

1.2 The Evaluation Function $f(n)$

The central innovation of the A* algorithm lies in its heuristic evaluation function, $f(n)$, which estimates the total cost of the path from the starting point through the current node n to the goal. This function is defined as the sum of two components: the true cost from the start, $g(n)$, and the estimated cost to the goal, $h(n)$.

$$f(n) = g(n) + h(n)$$

1.2.1 Path Cost ($g(n)$)

The function $g(n)$ represents the actual cost of the path taken from the initial starting node to the current node n. This is a known, objective value and is a direct measure of the work done so far. In traditional pathfinding, $g(n)$ might be the distance traveled;

in optimization problems like the one discussed in this report, $g(n)$ represents the cumulative loss differential between two consecutive node while traversing the search graph.

1.2.2 Heuristic Estimate ($h(n)$)

The function $h(n)$ is the heuristic estimate of the cost from the current node n to the final goal node. A heuristic is an educated guess that guides the search towards the most promising regions of the search space. The effectiveness and properties of the A* algorithm are highly dependent on the quality of this heuristic.

The heuristic $h(n)$ must satisfy the following critical property to guarantee optimality:

- **Admissibility:** A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, meaning for all nodes n, $h(n) \leq h^*(n)$, where $h^*(n)$ is the true minimum cost from n to the goal.

If an admissible heuristic is used, A* is guaranteed to find the optimal (lowest-cost) path, provided a path exists.

Consistency

The condition of consistency is a stronger requirement than admissibility. A heuristic $h(n)$ is consistent if, for every node n and every successor node n' connected by an edge with cost $c(n, n')$, the estimated cost from n to the goal is less than or equal to the cost of moving to n' plus the estimated cost from n' to the goal.

$$\textbf{Consistency Condition: } h(n) \leq c(n, n') + h(n')$$

Consistency implies admissibility, provided $h(\text{goal}) = 0$. When a consistent heuristic is used, A* is guaranteed to find the optimal path without needing to re-open nodes (re-examine a node already in the Closed Set), because the f-cost along any optimal path is monotonically non-decreasing.

- **Optimality Guarantee:** If the heuristic $h(n)$ is admissible (and edge costs are non-negative), A* is guaranteed to find the optimal (lowest-cost) path. If the heuristic is consistent, A* is also guaranteed to be optimal and more efficient, as it never needs to process a node more than once.

1.3 Algorithm Mechanics

A* operates by iteratively expanding nodes, maintaining two key data structures:

- **Open Set (or Frontier):** A priority queue containing nodes that have been generated but not yet fully explored. Nodes in the open set are prioritized based on their calculated $f(n)$ value.
- **Closed Set:** A set containing nodes that have already been fully expanded. This prevents the algorithm from revisiting and reprocessing the same node multiple times.

The core loop of the A* algorithm is as follows:

1. Initialize the Open Set with the starting node.
2. While the Open Set is not empty:
    - Select the node n from the Open Set with the lowest $f(n)$ value.
    - If n is the goal state, terminate the search.
    - Move n from the Open Set to the Closed Set.
    - Otherwise, generate all successors of n. For each successor s:
        - Calculate the cost of the path to s via n: $g_{\text{new}}(s) = g(n) + \text{cost}(n, s)$.
        - **Path Evaluation and Update:**
            * If s is in the Open Set, check if $g_{\text{new}}(s) < g(s)$. If true, update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and update its priority in the Open Set.
            * If s is in the Closed Set, check if $g_{\text{new}}(s) < g(s)$. If true, this means a cheaper path to s has been found. Update $g(s)$ to $g_{\text{new}}(s)$, re-calculate $f(s) = g(s) + h(s)$, and move s back from the Closed Set to the Open Set for re-expansion.
            * If s is not in either set, it is a newly discovered node. Set its parent to n, set $g(s) = g_{\text{new}}(s)$, calculate $f(s) = g(s) + h(s)$, and add it to the Open Set.

1.4 Properties of A*

The A* search algorithm is known for two key theoretical properties:

- **Completeness:** If a solution path exists from the start node to the goal node, A* is guaranteed to find it, provided the branching factor is finite and edge costs are non-negative.
- **Optimality:** A* is guaranteed to find the lowest-cost path if the heuristic function $h(n)$ is admissible. This makes it a preferred method for problems where minimizing the cost (or, in the context of this project, minimizing the training error) is paramount.

The adaptation of A* from a simple pathfinding tool to a machine learning optimization technique, as explored in this report, requires careful consideration of the state space, the definition of the path cost $g(n)$, and the design of an effective, possibly admissible heuristic $h(n)$.

1.5 Tree Search vs. Graph Search

The problem space solved by A* is fundamentally a graph, yet the search strategy can be executed either as a Tree Search or a Graph Search. The difference lies entirely in how the algorithm handles states (nodes) that can be reached via multiple distinct paths, a common occurrence in graphs with cycles or multiple connecting routes.

1.5.1 Tree Search A*

The Tree Search variant treats every path to a state as a unique node in the search tree.

- **Mechanism:** Does not utilize a Closed Set. A node is generated and expanded without checking if an identical state has been reached previously via a different path.
- **Efficiency:** Avoiding the memory overhead of the Closed Set, leads to the redundant expansion of identical states, severely impacting time efficiency, especially in graphs with many cycles or alternative routes.
- **Optimality:** Only guaranteed if and only if the heuristic function $h(n)$ is admissible.

1.5.2 Graph Search A*

The Graph Search variant is the standard implementation of A* because it explicitly tracks visited states to avoid redundant work.

- **Mechanism:** Utilizes both the Open Set and the Closed Set to ensure that, ideally, no state is expanded more than once.
- **Efficiency:** Significantly more time-efficient than Tree Search, as it avoids re-exploring entire subgraphs.
- **Optimality:**
  - If the heuristic $h(n)$ is **consistent**, a node is guaranteed to be reached via the optimal path when it is first placed in the Closed Set. Therefore, no node ever needs to be re-opened.
  - If $h(n)$ is only **admissible**, the optimal path to a state might be discovered after the state has already been placed in the Closed Set. This motivates the re-opening mechanism.

1.5.3 Implementation Choice

In the context of this project, which adapts A* for machine learning optimization, given the experimental nature of the heuristic $h(n)$ and the $g(n)$ cost definition based on training loss, neither strict **admissibility** nor **consistency** can be guaranteed. Therefore, the implementation utilizes the Graph Search A* algorithm, characterized by the explicit use of both the Open Set and the Closed Set. Crucially, the algorithm includes the necessary logic to re-open nodes from the Closed Set if a new, cheaper path to that node is discovered. This ensures the best possible path is found despite the non-ideal properties of the custom heuristic.

Chapter 2

The Multilayer Perceptron and Traditional Training Methods

2.1 Multilayer Perceptron (MLP) Architecture

The Multilayer Perceptron (MLP) is a fundamental class of feed-forward artificial neural networks. It is distinguished by having one or more *hidden layers* between the input layer and the output layer, which allows the network to learn complex, non-linear relationships between inputs and outputs.

2.1.1 Neuron Structure and Function

The basic unit of the MLP is the artificial neuron, which receives a set of inputs, each associated with a synaptic weight. The neuron performs two main operations:

1. **Weighted Aggregation:** It calculates the weighted sum of the inputs, to which a *bias* (β) is added:

$$z = \left(\sum_{i=1}^{m} w_i x_i \right) + \beta$$

   where x_i are the inputs, w_i are the weights, and m is the number of inputs.

2. **Activation Function:** It applies a non-linear activation function (σ) to the aggregated result to produce the neuron's output (a):

$$a = \sigma(z)$$

2.1.2 The Loss Function

Training an MLP is an optimization problem that aims to minimize the error between the network's predicted output and the desired target output. This error is quantified by the loss function.

2.2 The Standard Training Method: Backpropagation

Traditionally, the MLP is trained using the **Backpropagation** (error retro-propagation) algorithm, which is based on the *Gradient Descent* method.

2.2.1 Gradient Descent

Gradient Descent is an iterative algorithm that adjusts the network's weights to move in the direction where the cost function decreases most rapidly. The weight update rule is given by:

$$w_{i,\text{new}} = w_{i,\text{old}} - \eta \frac{\partial E}{\partial w_i}$$

where η is the *learning rate* and $\frac{\partial E}{\partial w_i}$ is the gradient, which is the partial derivative of the error with respect to the weight w_i.

2.2.2 Error Backpropagation

Backpropagation is the efficient mechanism used to calculate this gradient. It utilizes the chain rule of differential calculus, propagating the error (and thus the gradient) backward, from the output layer to the hidden layers and finally to the input layer. This allows for the determination of the exact contribution of each weight to the total error, making the optimization scalable for deep networks.

2.3 Motivation for the A* Approach

Despite its effectiveness, Backpropagation has several critical limitations that motivate the exploration of alternative optimization methods, such as A:

- **Local Minima:** Gradient Descent tends to get trapped in local minima or flat regions (plateaus) of the cost surface, preventing the network from achieving the global optimal solution.

- **Parameter Sensitivity:** The performance of Backpropagation is highly dependent on the correct selection of the η hyperparameter (learning rate). A value that is too high can cause oscillations, while a value that is too low makes training excessively slow.
- **Local Nature:** Backpropagation is a purely local algorithm, guided only by the gradient information at the current point. It lacks the global "vision" of the cost surface that an informed search algorithm like A* possesses.

This report proposes to frame the MLP training not as a continuous descent problem, but as a pathfinding problem in the quantize weight space, where A* can potentially overcome the local exploration limitations of Backpropagation, albeit at the expense of algorithmic training efficiency.

Chapter 3

The A* Search Framework for MLP Training

The primary objective of this project is to reformulate the optimization challenge of training a Multilayer Perceptron (MLP) as a **shortest-path problem** within a discrete, high-dimensional weight space. This chapter introduces the framework designed to achieve this, detailing how the continuous weight space is discretized, how the nodes and edges of the search graph are defined, and how the A* algorithm's evaluation function is adapted to guide the network towards optimal weights.

3.1 Discretization of the Weight Space

The A* algorithm fundamentally operates on a discrete graph. Therefore, the continuous domain of real-valued weight parameters must be converted into a finite, discrete set of possible states. This is handled by the `QuantizedMLP` class.

3.1.1 Quantized State Definition

To define the search nodes, the floating-point parameters (weights and biases) of the MLP are subjected to uniform quantization. Given a predefined **quantization factor** (Q), every weight w_i is rounded to the nearest multiple of $1/Q$.

$$w_{i,\text{quantized}} = \text{round}(w_i \cdot Q)/Q$$

The number of potential discrete weight configurations defines the size of the search space. To manage the complexity and avoid instability, the parameters are constrained to a defined numerical range $[\text{min\_range}, \text{max\_range}]$.

The total number of possible states is:

$$[\mathrm{Q}(\mathrm{min\_range} + \mathrm{max\_range}) + 1]^{N_{\mathrm{params}}}$$

3.1.2 State Representation and Hashing

In the A* implementation, the efficacy of the algorithm relies on quickly checking if a particular weight configuration has been visited before, or if a better path to that configuration has been found. This requires a unique, hashable identifier for each state.

The `QuantizedMLP` utilizes the `get_state_hash()` method, which concatenates all quantized parameters into a single, ordered tuple of integers (by multiplying the quantized weights by the quantization factor Q). This tuple serves as the key in the Closed Set (represented by the `g_costs` dictionary in the implementation) for tracking the minimum cost found so far to reach that state.

3.2 Search Node and Transition Definition

3.2.1 The Search Node

A single search node n is represented by the `SearchNode` class and contains:

1. The current `QuantizedMLP` configuration (a unique, quantized set of weights).
2. The calculated cost-to-come, $g(n)$.
3. The heuristic estimate to the goal, $h(n)$.
4. The total estimated cost, $f(n) = g(n) + h(n)$.

3.2.2 Defining Transitions (Neighbors)

The edges of the search graph are defined by the allowed transitions between weight configurations, generated by the `get_neighbors` function. In this structured approach, a transition represents a modification to a localized block of parameters.

A neighbor state s is generated from the current state n by applying a sliding window of size $H \times W$ for weight matrices (or a corresponding 1D window for bias vectors) across the layers. Every parameter $w_{i,j}$ falling within the boundaries of the window at a specific position $p = (p_x, p_y)$ is updated by exactly one quantization step:

$$w_{i,j,s} = w_{i,j,n} \pm \frac{1}{Q} \quad \forall (i,j) \in \text{Window}_p$$

The number of possible transitions (neighbors) per layer is determined by the total number of valid window positions in a matrix of size $M \times N$, given a horizontal stride x_s and vertical stride y_s:

$$\text{Slides} = \left(\lfloor \frac{M - H}{y_s} \rfloor + 1 \right) \times \left(\lfloor \frac{N - W}{x_s} \rfloor + 1 \right)$$

This structured transition model is adopted to provide several key benefits:

- **Exploitation of Spatial Locality:** By updating clusters of weights simultaneously, the search effectively navigates the functional block nature of neural network layers. This aligns with the intuition that neighboring weights often contribute to the same local feature representations.
- **Search Space Pruning:** Transitioning via kernels significantly reduces the branching factor of the search graph. This pruning of possible next states allows the algorithm to focus on regional adjustments that have a more cohesive impact on the network's output.
- **Computational Efficiency:** The structured approach leads to a substantial decrease in training time and memory consumption. By generating fewer but more meaningful neighbor states, the method minimizes the memory overhead typically associated with tracking massive neighborhood sets.
- **Scalability:** These performance gains are critical for applying the methodology to larger and deeper network architectures, effectively mitigating the curse of dimensionality inherent in high-dimensional weight spaces.

The total number of neighbors for any given state n is thus:

$$N_{\text{neighbors}} = 2 \cdot \sum_{\ell \in \text{Layers}} \text{Slides}_\ell$$

ensuring a controlled and computationally feasible traversal of the weight space.

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & w_{1,3}^{(l)} & w_{1,4}^{(l)} & w_{1,5}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & w_{2,3}^{(l)} & w_{2,4}^{(l)} & w_{2,5}^{(l)} \\ w_{3,1}^{(l)} & w_{3,2}^{(l)} & w_{3,3}^{(l)} & w_{3,4}^{(l)} & w_{3,5}^{(l)} \\ w_{4,1}^{(l)} & w_{4,2}^{(l)} & w_{4,3}^{(l)} & w_{4,4}^{(l)} & w_{4,5}^{(l)} \end{bmatrix}$$

Figure 3.1: Plot of the sliding window in position (1,1)

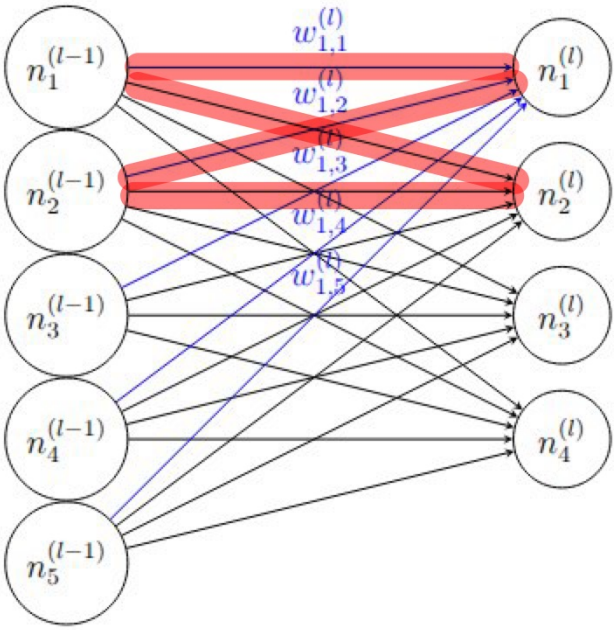

Figure 3.2: Plot of the connection involved in the sliding window positioned in (1,1)

3.3 A* Cost Function Adaptation for Training

The successful application of A* to optimization relies entirely on correctly defining the cost components $g(n)$ and $h(n)$ in the context of neural network training.

3.3.1 Heuristic Function $h(n)$: Current State Loss

In this formulation, the heuristic $h(n)$ directly represents the network's performance at node n and it is defined as the current loss value evaluated on the training data:

$$h(n) = L(n)$$

This heuristic prioritizes configurations that minimize the objective function, guiding the search toward lower-cost states in the weight landscape.

3.3.2 Path Cost $g(n)$: Accumulated Loss Differential

The cost-to-come, $g(n)$, represents the cumulative cost of the path taken from the initial configuration to the current node n. The cost of each transition (edge) is defined by the improvement or degradation in loss between the parent and the child node n.

The step cost Δg is defined as:

$$\Delta g = L(\text{n}) - L(\text{parent})$$

The total path cost is then updated iteratively:

$$g(n) = g(\text{parent}) + \Delta g$$

By defining Δg in terms of the loss differential, the path cost effectively tracks the total variation in loss encountered during the traversal of the search space.

3.3.3 Total Evaluation Function $f(n)$

The combined evaluation function $f(n)$ determines the priority of nodes in the search queue:

$$f(n) = g(n) + h(n) = (g(\text{parent}) + (L(n) - L(\text{parent}))) + L(n)$$

By selecting the node with the minimum $f(n)$, the algorithm seeks a path that minimizes both the absolute error of the current state $L(n)$ and the cumulative change in loss required to reach that state $g(n)$.

3.3.4 Admissibility Analysis

In standard A* search, a heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal, i.e., $h(n) \leq h^*(n)$. In this revised framework, the "goal" is implicitly the global minimum of the loss landscape ($L = 0$), and the cost is measured in units of loss differential.

1. **Definition:** Both the heuristic $h(n) = L(n)$ and the true cost $h^*(n)$ are expressed in the same units (loss). The true cost $h^*(n)$ represents the cumulative sum of loss differentials required to reach the global minimum starting from node n.

2. **The Admissibility Challenge:** For $h(n)$ to be admissible, the condition $h(n) \leq h^*(n)$ must hold. Under the current definitions, $h(n) = L(n)$, which is a non-negative value (the loss), while the true cost $h^*(n)$ is the sum of loss differentials: $h^*(n) = \sum_n^{goal} \Delta g$.

   In any scenario where the path toward the goal is monotonically improving, every step cost $\Delta g = L(n') - L(n)$ is strictly negative. Consequently, the true cost-to-go $h^*(n)$ becomes a summation of negative terms, yielding a negative total value. Because $L(n) \geq 0$, the inequality $L(n) \leq h^*(n)$ is fundamentally violated in every improving step of the training process.

   This implies that the heuristic is never admissible during a standard training descent. It provides a positive value (L) for a trajectory that accumulation a negative total cost ($\sum \Delta g$). Admissibility could only be restored if the search moved "uphill" into regions of significantly higher loss; however, since the primary goal is loss minimization, this violation is a permanent structural feature of the search design rather than an error.

3.3.5 Consistency Analysis

A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$, where $c(n, n')$ is the cost of the transition. With our new definitions:

- $h(n) = L(n)$

- $c(n, n') = \Delta g = L(n') - L(n)$
- $h(n') = L(n')$

Substituting these into the consistency inequality:

$$L(n) \leq (L(n') - L(n)) + L(n') \implies L(n) \leq 2L(n') - L(n) \implies 2L(n) \leq 2L(n')$$

Simplifying this, the **Consistency Condition** becomes:

$$L(n) \leq L(n')$$

Analysis of Scenarios w.r.t. Consistency

Unlike the previous version, consistency is now tied directly to the direction of the loss change ΔL:

1. **Scenario 1: Loss Decrease ($\Delta L < 0$):** When the sliding window update improves the model, $L(n') < L(n)$. In this case, the consistency condition $L(n) \leq L(n')$ is **violated**. This is a mathematically interesting result: every "good" step in training (one that reduces loss) technically violates the A* consistency property under these definitions.
2. **Scenario 2: Loss Increase ($\Delta L > 0$):** If the update moves into a worse region of the landscape, $L(n') > L(n)$. Here, the consistency condition is **satisfied**. This implies that the search is only "consistent" when it is moving away from the solution.
3. **Scenario 3: Stationary Loss ($\Delta L = 0$):** In plateaus where $L(n') = L(n)$, the heuristic is perfectly consistent.

By leveraging these efficiencies, the kernel-based method mitigates the "curse of dimensionality," making it possible to apply discrete search-based optimization to complex neural network structures that would otherwise be computationally prohibitive.

3.4 The A* Training Loop Implementation

The core training logic is contained within the `Trainer` class, which manages the A* search:

1. **Initialization:** The process begins by creating an `initial_node` from the starting MLP weights. This node is placed in the priority queue (`open_set`).

2. **Iteration:** In each iteration, the node with the lowest $f(n)$ is retrieved from the `open_set`.

3. **Optimality Check:** Before expanding the node, the current path cost $g(n)$ is checked against the minimum cost previously recorded for that state in `g_costs` to ensure that A* always processes the optimal path to a given state first.

4. **Expansion:** Neighbors are generated by the transition rule ($\pm 1/Q$ change to a weight subset).

5. **Cost Update:** For each neighbor, the new g and h values are calculated. If the new path to the neighbor results in a lower cost-to-come ($g_{\text{new}} < g_{\text{old}}$), the neighbor is either updated in the search space or added to the `open_set`.

6. **Termination:** The search terminates when either the best node retrieved from the `open_set` satisfies the goal condition ($h(n) = 0$) or the maximum number of iterations is reached.

This framework provides a global, informed search mechanism for MLP training, aiming to overcome the local minimum issues inherent in traditional gradient-based methods like Backpropagation.

Chapter 4

Experimental Methodology and Results

This report conducts a comparative analysis of two optimization methodologies, focusing on their respective strengths and weaknesses.

The primary objective of this study is to analyze a novel approach using A* search principles, experimentally comparing its performance and behavior against the well-established optimization methodology of Gradient Descent. Furthermore, a crucial element of this comparison involves a detailed investigation into the A* framework's sensitivity to its core parameters; a hyperparameter finetuning process has been performed to understand their influence on achieving superior performance.

4.1 Experimental Methodology

To systematically evaluate the proposed A* search framework, a three-stage comparative protocol was designed and applied uniformly to sine dataset. This approach allows for a rigorous analysis of the A* algorithm's behavior under different hyperparameter settings and a direct performance comparison with a conventional gradient descent method.

4.1.1 MLP Architecture and Evaluation

For all experiments, a Multilayer Perceptron (MLP) architecture suitable for the complexity of the dataset was chosen. The loss function $L(n)$ for both A* evaluation (heuristic $h(n)$) and the Backpropagation baseline was chosen based on the task.

4.1.2 Three-Stage Comparative Protocol

Stage 1: Hyperparameter Grid Search and Sensitivity Analysis

The A* search performance is critically dependent on the discretization of the weight space. Therefore, the first stage involved an exhaustive grid search to:

1. Identify the optimal configuration of parameters that maximizes convergence speed and solution quality.
2. Understand the sensitivity of the A* search to changes in the search space.

The key hyperparameters subjected to optimization were:

- **Maximum Iterations (max_iterations):** The upper limit of the number of total node expansions.
- **Quantization Factor (Q):** Defining the granularity of the weight space ($1/Q$).
- **Parameter Range:** The boundary $[\text{min\_range}, \text{max\_range}]$ for the weights.
- **Kernels Shape and Strides:** This parameter set defines the window dimensions ($H \times W$) for the weight and bias matrices, alongside the horizontal (x_s) and vertical (y_s) strides.

Stage 2: A* Training with Optimal Configuration

The best-performing hyperparameter set from Stage 1 was selected and used to train the final MLP model. This training run recorded the total number of node expansions, the path taken (sequence of weight states), and the overall wall-clock time required for the optimization.

Stage 3: Backpropagation Baseline and Comparison

The A* performance was compared against the standard Stochastic Gradient Descent algorithm with Backpropagation. For a fair comparison, the SGD baseline was trained using its own optimal learning rate (η) derived from preliminary tuning. The final comparison focused on three critical metrics:

- **Convergence Speed:** Total A* node expansions vs. total SGD epochs required to achieve the minimum loss.

- **Final Loss Value:** The best loss achieved during training for both methods.
- **Wall-Clock Time:** Comparison of total training time.

4.2 Noisy Sine Function

The first experiment applies the A* training framework to a non-linear regression problem, assessing its ability to learn complex continuous functions.

4.2.1 Dataset Generation and Objective

The synthetic dataset was generated using the one-dimensional sine function with additive Gaussian noise following the Normal distribution.

- **Input (x):** 1,000 evenly spaced samples in the range $x \in [0, 4\pi]$.
- **Output (y):** Generated as $y = \sin(x) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, 0.1)$.
- **MLP Architecture:** Input (1 neuron) $\rightarrow$ Hidden Layer 1 (8 neurons, $ReLU$) $\rightarrow$ Hidden Layer 2 (16 neurons, $ReLU$) $\rightarrow$ Hidden Layer 3 (8 neurons, $ReLU$) $\rightarrow$ Output (1 neuron, $tanh$).

The network's task is to learn the underlying $\sin(x)$ mapping, minimizing the Mean Squared Error loss.

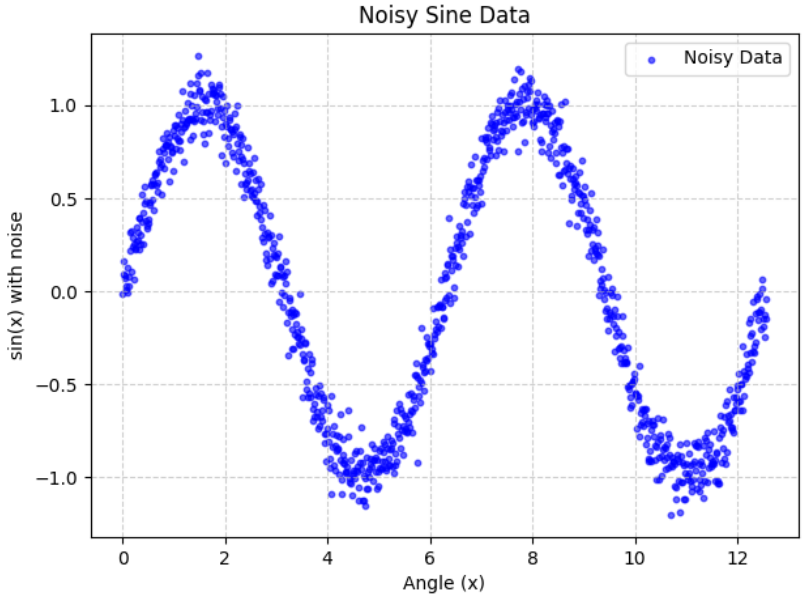

Figure 4.1: Plot of the noisy sine dataset

4.2.2 Stage 1: Grid Search Findings

Except for the specific parameter under evaluation during the grid search, the system is initialized with the following default hyperparameters:

- **Quantization Factor (Q):** 10
- **Parameter Range:** $[-20, 20]$
- **Kernel Geometry:** Weight kernels of size 3×3 and bias kernels of size 3.
- **Traversal Strides:** Horizontal (x_s) and vertical (y_s) strides are both set to 2.
- **Search Constraints:** A maximum of 2000 iterations.

Impact of the Quantization Factor
Based on the empirical results presented in the accompanying figure, the hyperparameter grid search demonstrated a clear dependency of the final achieved loss on the selected quantization factor, Q. Specifically, among the tested values ($Q = 10$, $Q = 100$, and $Q = 1000$), the lower and intermediate factors of $Q = 10$ and $Q = 100$ yielded the most favorable and comparable outcomes, both achieving a final mean loss of approximately 0.17.

In contrast, the highest quantization factor, $Q = 1000$, resulted in significantly poorer performance, with a final mean loss of 0.41. This suggests that while $Q = 10$ and $Q = 100$ provide a sufficiently discretized weight space for efficient A* traversal, $Q = 1000$ leads to a search space that is too fine-grained. At this higher resolution, the distance between meaningful states may hinder the algorithm's ability to reach a global minimum within the allocated iteration budget.

To evaluate the stability and convergence of the proposed methodology, the following results represent the average loss across 10 independent runs for each tested hyperparameter configuration. Figure below provides a clear measure of statistical variability and reproducibility, where the solid lines represent the average loss and the surrounding shaded regions indicate the one standard deviation ($\pm 1\sigma$) interval.

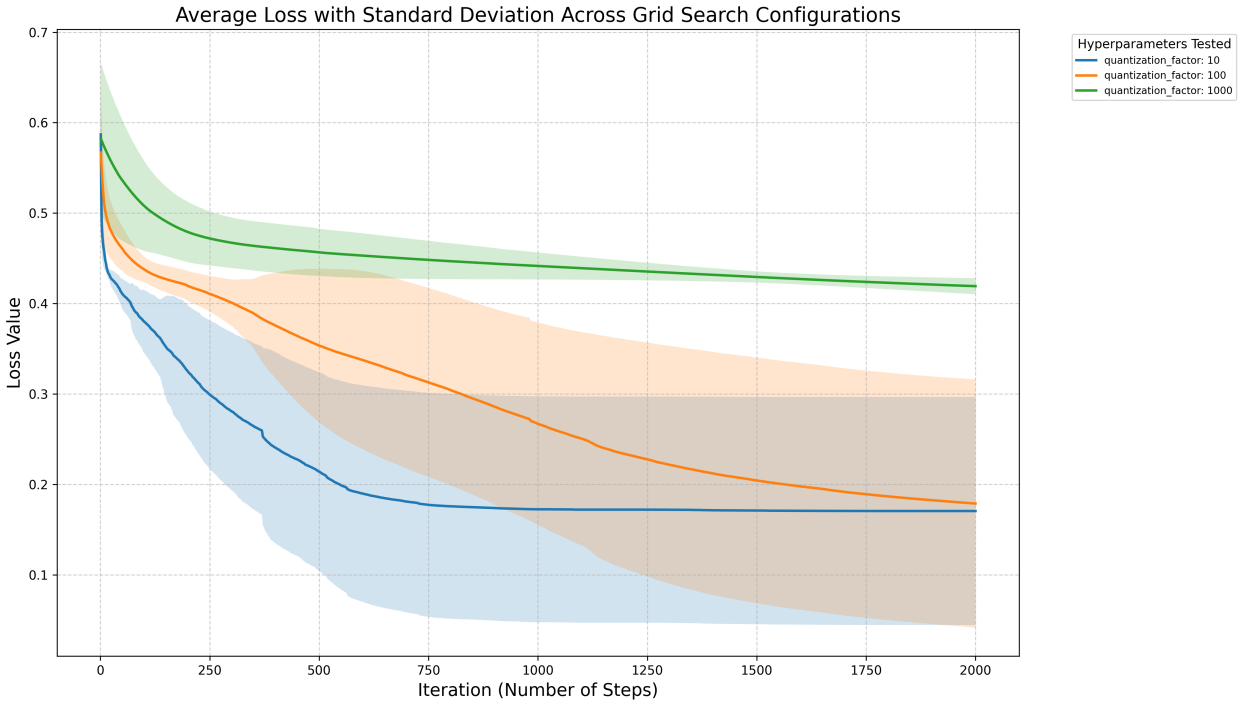

Figure 4.2: Comparison of the mean loss across training iterations for varying quantization factors ($Q \in \{10, 100, 1000\}$)

Table 4.1: Impact of Quantization Factor (Q) on Sine Regression Performance

QF (Q)	**Mean Loss**	**Median Loss**	**Std. Dev.**	**Min Loss**	**Max Loss**	**Mean Time (s)**
10	**0.1705**	**0.1049**	0.1261	0.0570	0.4328	**367.11**
100	0.1788	0.1093	0.1373	**0.0435**	**0.3834**	423.68
1000	0.4192	0.4222	**0.0088**	0.4018	0.4304	431.09

Impact of the Network Parameters Range

A second set of experiments was conducted to determine the optimal numerical weight range for the network parameters, defining the boundary constraints for the quantized weights. We evaluated three distinct intervals: $[-5, 5]$, $[-10, 10]$, and $[-20, 20]$.

Analysis of the results demonstrates a non-monotonic trend in performance relative to the range width. The intermediate range of $[-10, 10]$ yielded the best performance, achieving the lowest mean loss of 0.1672. In comparison, the narrowest range ($[-5, 5]$) resulted in a higher mean loss of 0.2507, likely due to insufficient model expressiveness. Interestingly, the widest range ($[-20, 20]$) also showed a performance degradation compared to the medium interval, with a mean loss of 0.2100.

These observations suggest that while a range must be broad enough to provide the necessary search capacity, an excessively wide weight space may increase the search complexity. Consequently, the $[-10, 10]$ interval strikes the most effective balance between parameter flexibility and search efficiency for this architecture.

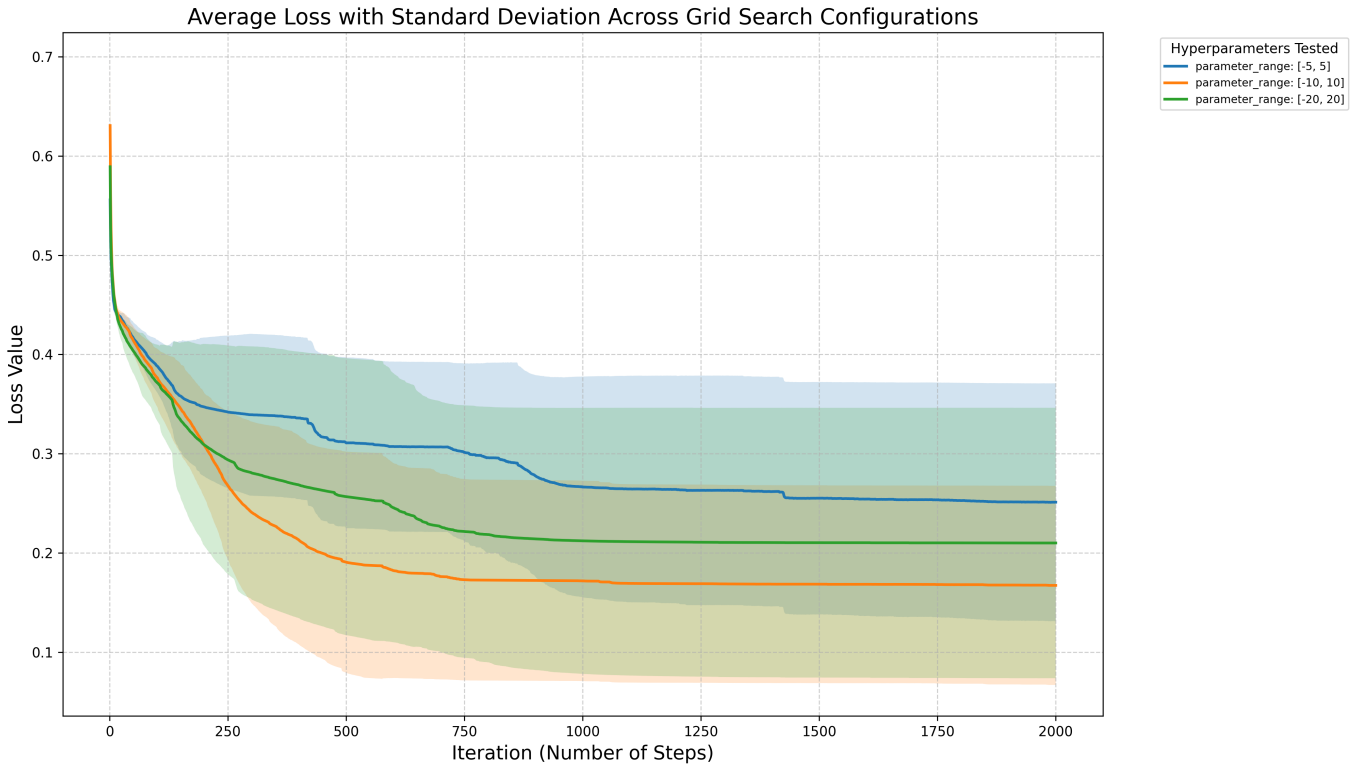

Figure 4.3: Comparison of the mean loss across training iterations for varying Parameter Ranges

Table 4.2: Impact of Parameter Range (PR) on Sine Regression Performance

PR	Mean Loss	Median Loss	Std. Dev.	Min Loss	Max Loss	Mean Time (s)
$[-5, 5]$	0.2507	0.2666	0.1200	0.0599	0.4259	**274.99**
$[-10, 10]$	**0.1672**	**0.1456**	**0.1003**	0.0618	0.3893	312.69
$[-20, 20]$	0.2100	0.2104	0.1363	**0.0433**	**0.3717**	369.17

Effect of Maximum Iterations ($I_{\max}$)

The third parameter evaluated was the maximum number of search iterations ($I_{\max}$), testing values of $1,000$ and $2,000$. Unlike fixed-step methodologies, in this framework, $I_{\max}$ acts as a hard cap on the exploration budget within the weight space.

As demonstrated by the empirical results, the configuration with $I_{\max} = 1,000$ iterations proved to be the most efficient, achieving a mean terminal loss of 0.1789. Interestingly, doubling the computational budget to $I_{\max} = 2,000$ did not yield a performance gain, instead resulting in a slightly higher mean loss of 0.1884 and a significant increase in average execution time from approximately 201 seconds to 348 seconds.

This observation suggests that the search typically converges to a stable local minimum within the first $1,000$ iterations. Rather than facilitating the discovery of deeper minima, the extended budget allows the algorithm to persist in high-dimensional plateaus without meaningful improvement.

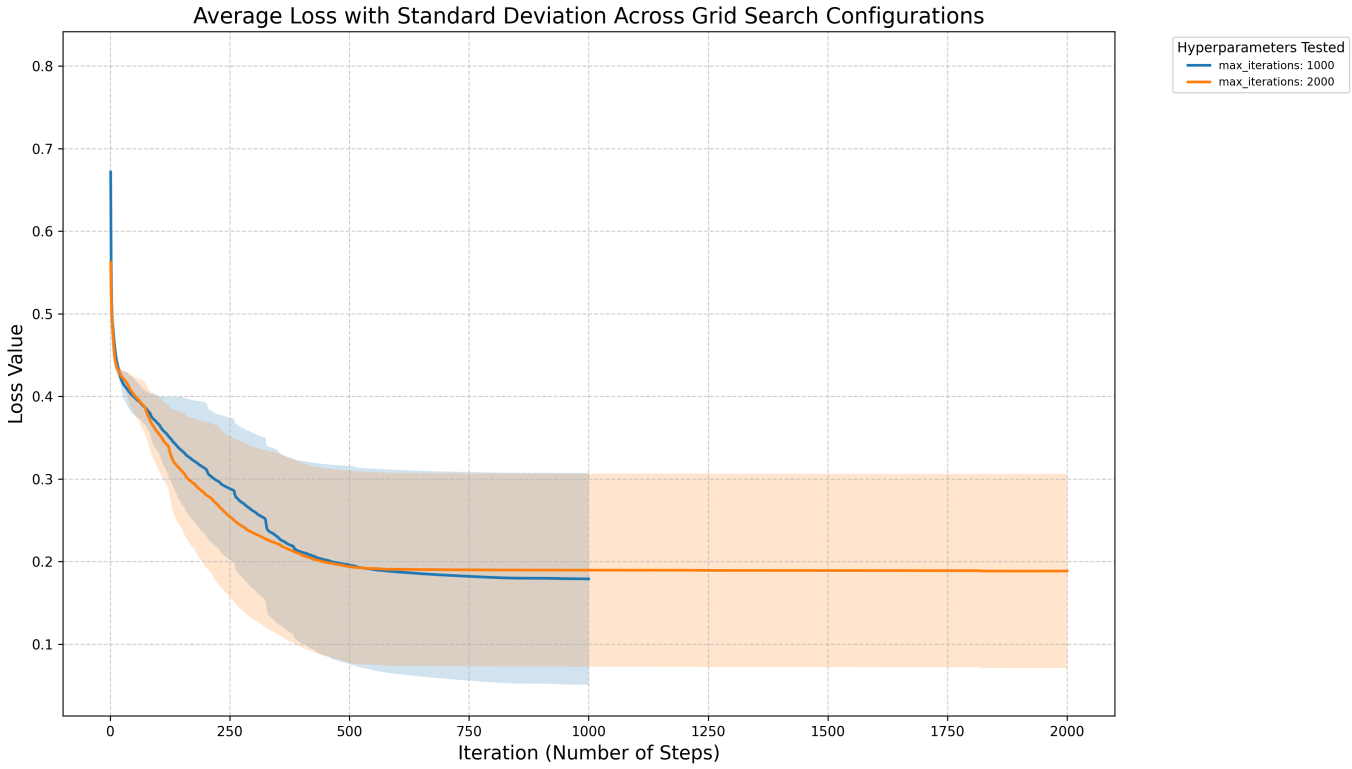

Figure 4.4: Comparison of the mean loss across training iterations for varying $I_{\max}$ parameter

Table 4.3: Impact of Maximum Iterations on Sine Regression Performance

Iterations	Mean Loss	Median Loss	Std. Dev.	Min Loss	Max Loss	Mean Time (s)
1000	**0.1789**	**0.1360**	0.1283	0.0380	0.3563	**201.29**
2000	0.1884	0.1591	0.1176	0.0566	0.3546	347.74

Effect of Kernel Geometry and Traversal Strides

A critical set of experiments was conducted to evaluate the influence of the sliding window geometry on search efficiency. We tested four configurations varying the Weight Kernel size (WK), Bias Kernel size (BK), and the uniform traversal stride (S), where horizontal and vertical strides were kept equal ($x_s = y_s = S$). These parameters determine the search graph's branching factor and the 'spatial span' of each transition, effectively governing the extent of weight space coverage per exploration step.

Analysis of the results indicates that the choice of kernel dimensions significantly impacts the convergence rate and terminal loss. Larger kernels effectively "squash" the search depth by updating more parameters per hop, while the stride regulates the overlap between successive updates.

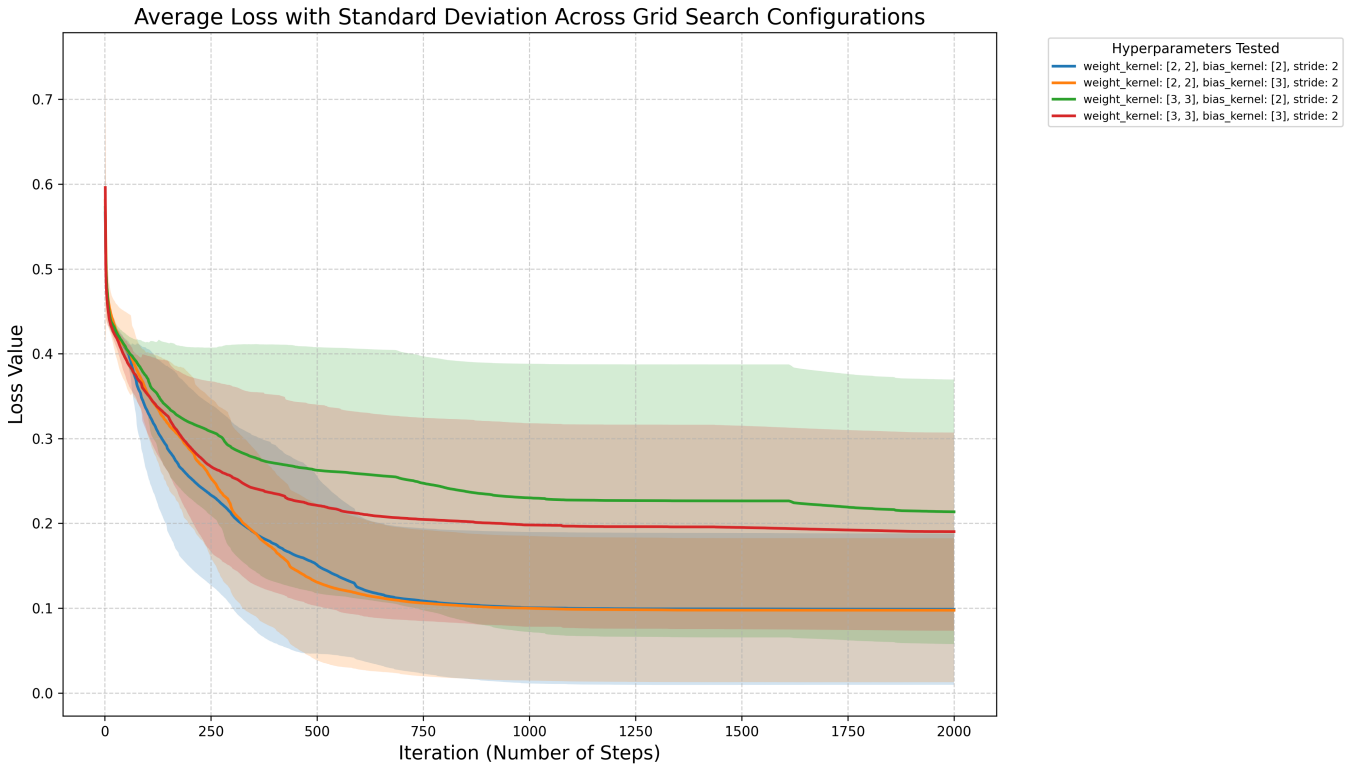

Figure 4.5: Evolution of mean loss across iterations for varying kernel geometries and stride values

Table 4.4: Impact of Kernel Sizes and Strides on Sine Regression Performance

WK	BK	S	Mean Loss	Median Loss	Std. Dev.	Min Loss	Max Loss	Mean Time (s)
[2, 2]	[2]	2	0.0987	**0.0565**	0.0892	**0.0196**	**0.3134**	499.99
[2, 2]	[3]	2	**0.0975**	0.0673	**0.0848**	0.0217	0.3199	478.15
[3, 3]	[2]	2	0.2136	0.1644	0.1560	0.0505	0.4360	395.87
[3, 3]	[3]	2	0.1902	0.1706	0.1168	0.0673	0.3973	**357.17**

This structured traversal ensures that the search focuses on localized functional blocks, prioritizing regional weight adjustments. By optimizing these geometric parameters, the algorithm achieves a balance between granular precision and broad exploration, mitigating the computational overhead typically associated with discrete search in high-dimensional spaces.

4.2.3 Stage 2 & 3: Training and Comparative Analysis

Explanation of Plots: Statistical Methodology
The comparative analysis of A* search and Gradient Descent relies on two complementary visualization techniques, each providing a distinct view of the algorithms' performance across the ten independent runs.
Mean Loss with Standard Deviation Shading
The solid line represents the average loss ($\bar{L}$) across all ten runs at every iteration, clearly demonstrating which method achieves faster convergence or reaches a lower loss plateau on average.
Interpretation of Variability (Standard Deviation $\pm 1\sigma$): The shaded area surrounding the mean line represents the standard deviation ($\pm 1\sigma$) of the loss at each iteration. This is a crucial measure of an algorithm's consistency and robustness.

- A narrow shaded band means the method is highly consistent, its performance is reliable and typically near the average, regardless of the random initial weights.
- A wide shaded band means the method is volatile (σ is large) and highly dependent on the random initialization. A wide band often signals that the algorithm frequently gets stuck in poor local minima or conversely, occasionally finds good solutions.

Final Loss Box Plot
This plot provides a concise summary of the distribution of the final terminal loss achieved by each method after the completion of $I_{\max}$ iterations. It is ideal for comparing the final state quality.
Interpretation of Distribution:

- **The Median Line:** The horizontal line inside the box shows the median (the 50th percentile) result. The median is a robust measure of typical performance, as it is less sensitive to extreme outliers than the mean.
- **The Box (Interquartile Range, IQR):** The vertical span of the box represents the middle 50% of your results (from the 25th to the 75th percentile). A very short box indicates high clustering and consistency in the results. If one box is much lower than another, that method is definitively superior in its typical performance range.
- **Whiskers and Dots (Range and Outliers):** The whiskers show the full range of results (excluding outliers), and individual dots beyond the whiskers identify extreme outliers. These elements help to see how often a method results in a failed run or a single exceptional result.

Calculation of Whisker Range and Outliers
The length of the whiskers is not arbitrary; it is statistically determined by the data's central spread (the IQR) to identify extreme values (outliers).

- **Interquartile Range (IQR):** This is the length of the box, calculated as $IQR = Q_3 - Q_1$.
- **Whisker Boundaries:** The upper and lower whiskers extend to the most extreme data points that are no more than $\mathbf{1.5 \times IQR}$ away from the box edges. The boundaries are formally defined as:

$$\text{Upper Bound} = Q_3 + 1.5 \times IQR$$

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR$$

- **Outliers:** Any final loss value that falls outside these calculated bounds is considered an outlier and is plotted as an individual point (dot). In the context of loss, points above the upper whisker represent runs where the algorithm performed significantly worse than its typical operation.

4.2.4 Comparison with Gradient Descent: $I_{\max} = 1,000$

To evaluate the mid-term behavior of both A* search and Gradient Descent, the comparative study was conducted using a computational budget of $I_{\max} = 1,000$ steps. Ten independent runs ($n = 10$) were executed for each method to ensure statistical relevance. The results are summarized in Table 4.5, followed by an analysis of the performance metrics.

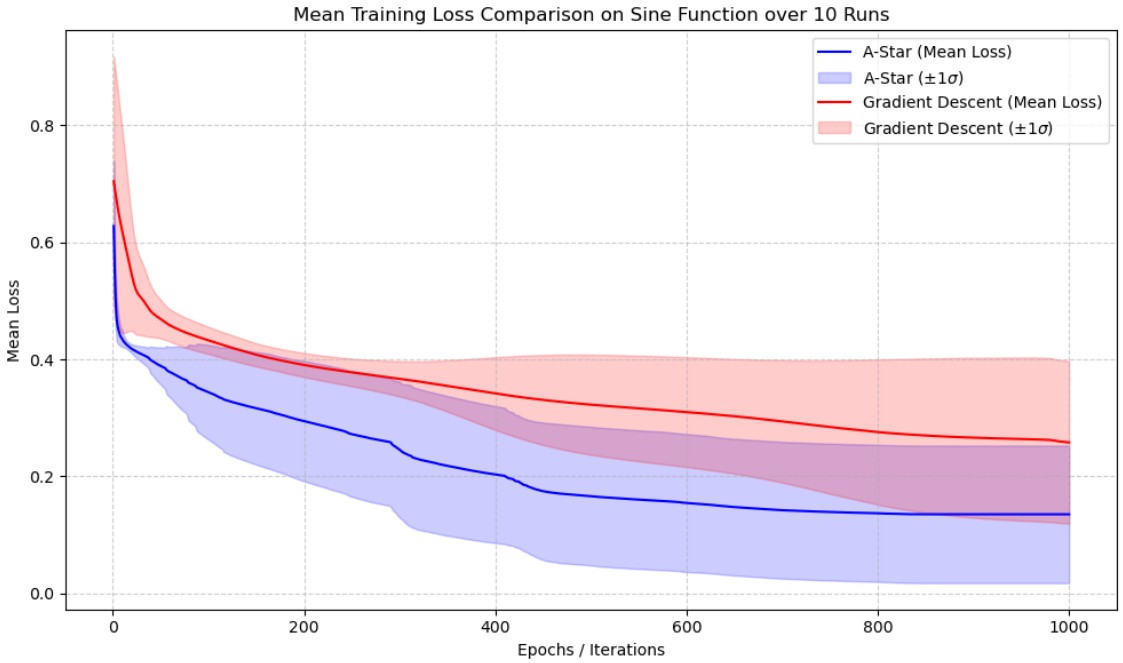

Figure 4.6: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 1{,}000$ Iterations.

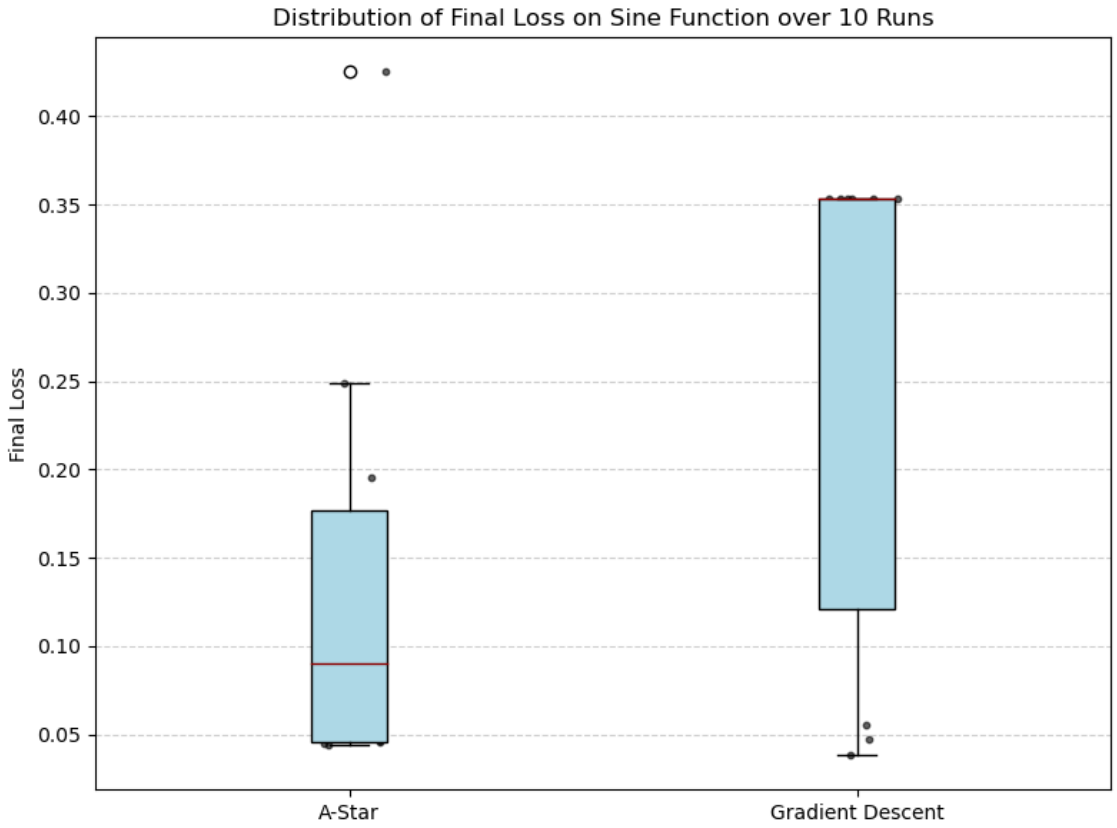

Figure 4.7: Final Loss Box Plot for $I_{\max} = 1{,}000$ Iterations.

Table 4.5: Comparative Statistics Over Ten Runs for $I_{\max} = 1,000$ Iterations

Metric	**A* Search**	**Gradient Descent**
Average Loss ($\bar{L}$)	**0.135254**	0.257953
Median Loss	**0.090426**	0.353416
Standard Deviation (σ)	**0.117442**	0.138664
Variance (σ^2)	**0.013793**	0.019228
Minimum Loss ($L_{\min}$)	0.044036	**0.038053**
Maximum Loss ($L_{\max}$)	0.425110	**0.353651**
Average Training Time (s)	281.61	**12.69**

Statistical and Trajectory Analysis

Dominance in Performance: At the $1,000$ iteration, the A* Search algorithm demonstrates a clear lead in terms of central tendency. The average loss for A* ($\bar{L} = 0.1353$) is significantly lower than that recorded for Gradient Descent ($\bar{L} = 0.2580$). The disparity in Median Loss (0.0904 for A* vs. 0.3534 for GD) suggests that, within this specific iteration window, the heuristic approach is more effective at converging rapidly toward low-loss states, whereas GD appears to suffer from slower convergence.

Convergence and Best-Case Performance: When examining the peak potential of both methods, the results appear highly competitive. Gradient Descent achieved the absolute Minimum Loss ($L_{\min} = 0.0381$), narrowly surpassing the best performance of A* ($L_{\min} = 0.0440$). However, the proximity between the average and the minimum in the A* results indicates that the latter identifies high-quality solutions with greater frequency, while GD despite hitting the lowest point in a single run presents a distribution of results less centered toward the optimum.

Reliability and Variance: In terms of consistency, A* Search demonstrates superior stability in this scenario. With a standard deviation of $\sigma = 0.1174$ (lower than the 0.1387 of GD), the Novel method shows less dispersion across different runs. This suggests that the heuristic-guided exploration is less influenced by stochasticity or initial conditions compared to the continuous nature of gradient updates, ensuring a more predictable output.

Computational Efficiency: The gap in computational efficiency remains the most critical factor. Gradient Descent completed the training in an average of only 12.69 seconds, while A* required 281.61 seconds. This approximately $22\times$ advantage in favor of the classic method highlights the computational overhead inherent in priority queue management and node expansion typical of A*, contrasted with the simplicity of analytical weight updates in GD.

Conclusion for $I_{\max} = 1,000$: At $1,000$ iterations, A* Search emerges as the better-performing option, offering superior average loss and statistical stability compared to Gradient Descent. Although GD maintains an insurmountable advantage in execution speed, A*'s ability to identify higher-quality solutions within a limited number of iterations confirms the strong potential of this heuristic framework as a robust alternative to traditional optimization methods.

4.2.5 Comparison with Gradient Descent: $I_{\max} = 2,000$

To evaluate the mid-term behavior of both A* search and Gradient Descent, the comparative study was conducted using a computational budget of $I_{\max} = 2,000$

steps. Ten independent runs ($n = 10$) were executed for each method to ensure statistical relevance. The results are summarized in Table 4.6, followed by an analysis of the performance metrics.

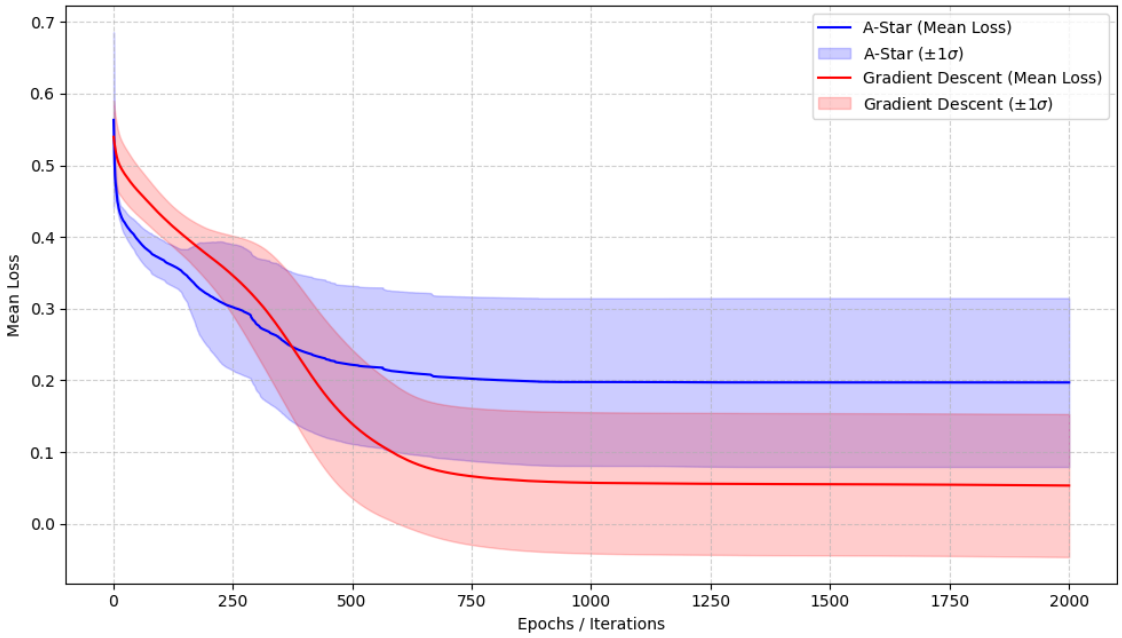

Figure 4.8: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}000$ Iterations.

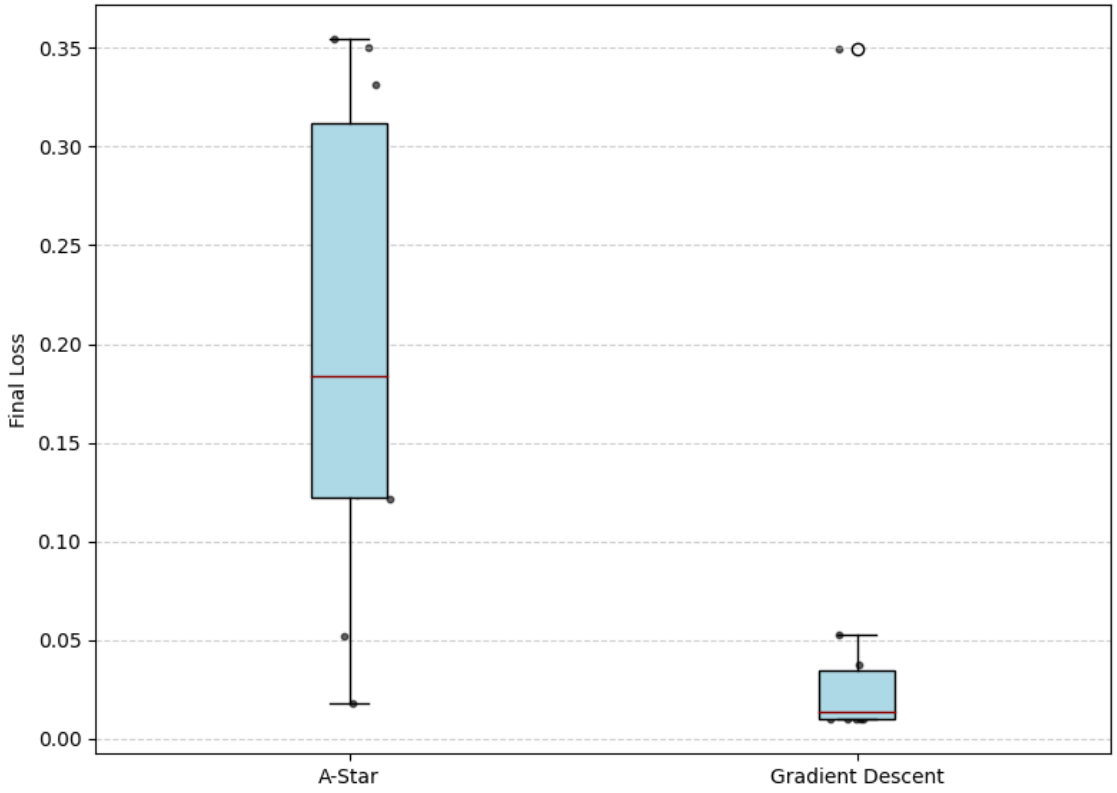

Figure 4.9: Final Loss Box Plot for $I_{\max} = 2{,}000$ Iterations.

Table 4.6: Comparative Statistics Over Ten Runs for $I_{\max} = 2,000$ Iterations

Metric	**A* Search**	**Gradient Descent**
Average Loss ($\bar{L}$)	0.197100	**0.053283**
Median Loss	0.183644	**0.013862**
Standard Deviation (σ)	0.117757	**0.099745**
Variance (σ^2)	0.013867	**0.009949**
Minimum Loss ($L_{\min}$)	0.017940	**0.009764**
Maximum Loss ($L_{\max}$)	0.354409	**0.349630**
Average Training Time (s)	453.53	**18.07**

Statistical and Trajectory Analysis

Dominance in Performance: At the $2,000$ iteration mark, Gradient Descent exhibits a clear performance lead in terms of central tendency. The average loss for GD ($\bar{L} = 0.0533$) is significantly lower than that of A* ($\bar{L} = 0.1971$). The disparity in Median Loss (0.0139 for GD vs. 0.1836 for A*) further suggests that GD more frequently reaches a high-precision state within this specific computational window.
Convergence and Best-Case Performance: When examining the peak potential of both algorithms, the results are remarkably competitive. GD achieved a Minimum Loss of $L_{\min} = 0.0098$, while A* followed closely with $L_{\min} = 0.0179$. This comparison indicates that both methods are capable of identifying high-quality solutions; while GD's continuous gradient exploitation gives it a slight edge in refinement, A* demonstrates that its heuristic search can successfully navigate to the same neighborhood of the global minimum.
Reliability and Variance: The consistency of the two methods is quite comparable at this scale. GD maintains a Standard Deviation of $\sigma = 0.0997$, which is only slightly lower than the $\sigma = 0.1178$ recorded for A*. This narrow margin suggests that both algorithms offer a similar level of reliability across independent runs. The small difference indicates that A* remains a robust contender in terms of stability.
Computational Efficiency: There remains a significant difference in computational overhead. GD completed the training in an average of 18.07 seconds, whereas A* required 453.53 seconds. This $25\times$ speed advantage for the classic approach highlights the current efficiency of gradient-based updates compared to the more memory-intensive node expansion process inherent in A* search.
Conclusion for $I_{\max} = 2,000$: At $2,000$ iterations, Gradient Descent proves to be highly efficient, offering a lower average loss and faster execution. However, the A* method remains a novel alternative; it achieves a minimum loss and a level of statistical consistency (standard deviation) that are comparable with the classic approach. While A* currently requires more time to process its search space, its ability to reach competitive high-quality solutions suggests significant potential for the heuristic-driven framework.

4.3 Iris Flower Classification

The second experiment applies the A* training framework to a classic classification task, assessing its ability to perform MLP parameter optimization in a well known dataset.

4.3.1 Dataset and Objective

The experiment utilizes the canonical Iris dataset, which consists of 150 samples of three species of Iris flowers (Setosa, Versicolor, Virginica), based on four key features.

- **Input (x):** 4 features (sepal length, sepal width, petal length, petal width) standardized to a common scale.
- **Output (y):** Three classes (species), encoded using one-hot encoding (e.g., $[1, 0, 0]$ for Setosa).
- **MLP Architecture:** Input (4 neurons) $\rightarrow$ Hidden Layer 1 (8 neurons, ReLU) $\rightarrow$ Hidden Layer 2 (16 neurons, ReLU) $\rightarrow$ Hidden Layer 3 (8 neurons, ReLU) $\rightarrow$ Output (3 neurons).

The network's task is a three class classification, aiming to learn the mapping from the four physical measurements to the correct species, minimizing the Categorical Cross-Entropy loss. The performance comparison between A* and Gradient Descent will be primarily evaluated based on this loss function.

4.3.2 Stage 1: Optimal Hyperparameters

The optimal hyperparameters found during the sine experiment were adopted for the final comparison phase of this dataset:

- **Quantization Factor:** $Q = 10$
- **Parameter Range:** $[-10, 10]$
- **Kernels Shape and Stride:** Weight_kernel $= (2, 2)$, Bias_kernel $= (2, 1)$ and Stride $= 2$

4.3.3 Stage 2 & 3: Training and Comparative Analysis

The methodology for visualization and statistical analysis remains identical to that employed for the sine dataset. The comparison between A* and Gradient Descent is performed over ten independent runs ($n = 10$) for two iteration budgets: $I_{\max} = 1,000$ and $I_{\max} = 2,000$. The same Mean Loss with Standard Deviation Shading and Final Loss Box Plot visualizations are utilized.

4.3.4 Comparison with Gradient Descent: $I_{\max} = 1,000$

The comparative study between A* and GD was first executed with a budget of $I_{\max} = 1,000$ steps. The statistical results are summarized in Table 4.7, and the performance trajectory is presented in the following figures.

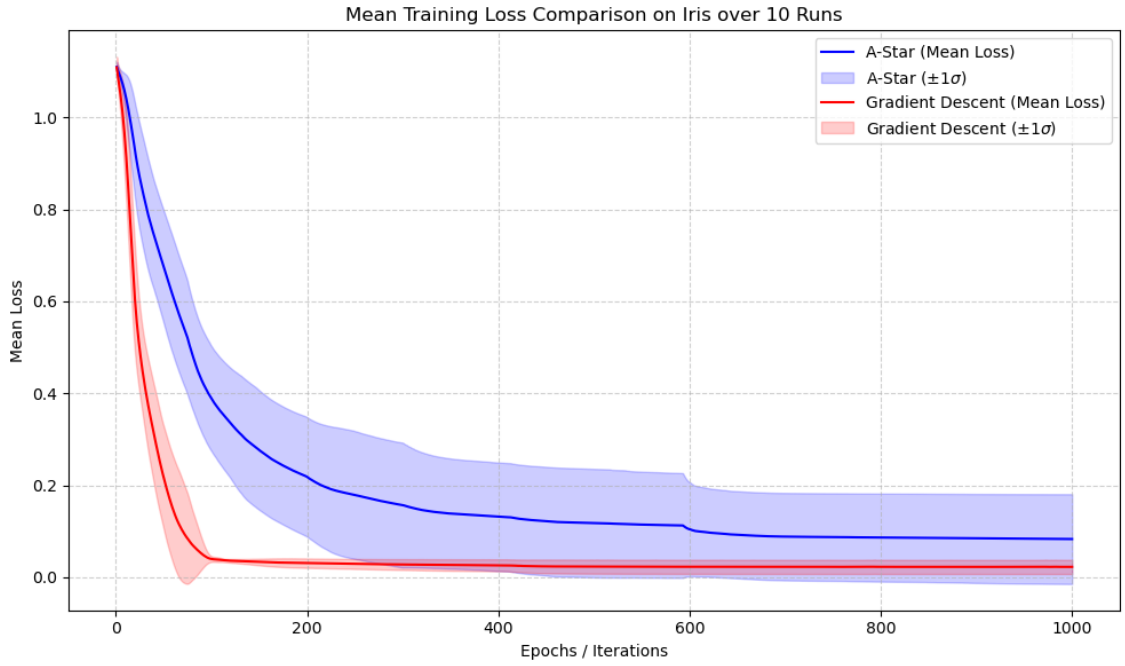

Figure 4.10: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 1{,}000$ Iterations

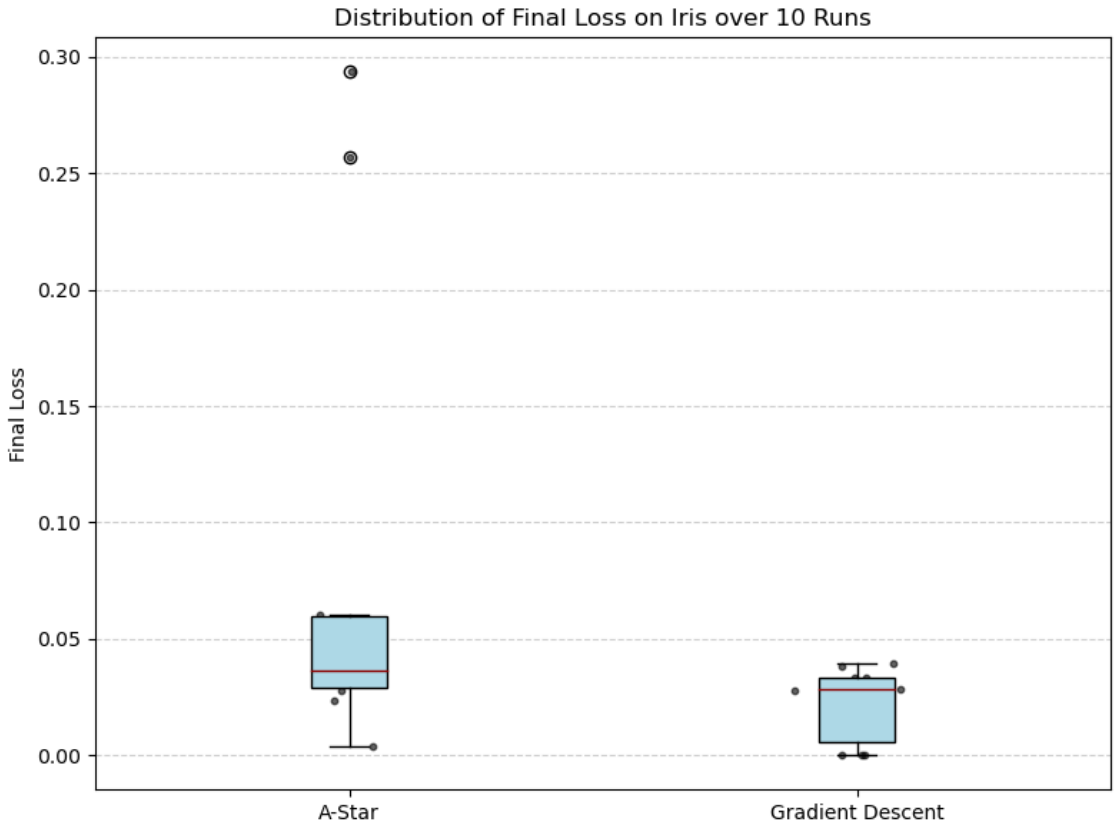

Figure 4.11: Final Loss Box Plot for $I_{\max} = 1{,}000$ Iterations

Table 4.7: Comparative Statistics Over Ten Runs for $I_{\max} = 1,000$ Iterations

Metric	**A* Search**	**Gradient Descent**
Average Loss ($\bar{L}$)	0.082992	**0.022390**
Median Loss	0.036709	**0.028436**
Standard Deviation (σ)	0.097638	**0.015363**
Variance (σ^2)	0.009533	**0.000236**
Minimum Loss ($L_{\min}$)	0.003700	**0.000017**
Maximum Loss ($L_{\max}$)	0.293478	**0.039812**
Average Training Time (s)	359.86	**4.50**

Statistical and Trajectory Analysis

Dominance in Performance: At the $1,000$ iteration on the Iris dataset, Gradient Descent maintains a consistent lead in terms of central tendency. The average loss for GD ($\bar{L} = 0.0224$) is notably lower than that of A* ($\bar{L} = 0.0830$). Interestingly, the Median Loss values are much closer (0.0367 for A* vs. 0.0284 for GD), suggesting that while GD is more consistent on average, A* is frequently capable of reaching similar performance levels in the majority of its runs.

Convergence and Best-Case Performance: Both algorithms demonstrate the ability to identify high-quality solutions even with a restricted iteration budget. GD achieved a Minimum Loss of $L_{\min} = 0.000017$, while A* reached a remarkably low $L_{\min} = 0.0037$. This indicates that the heuristic search within the A* framework is effective at navigating the Iris loss landscape toward global minima.

Reliability and Variance: There is a visible difference in the stability of the two methods. GD exhibits very low variance ($\sigma^2 = 0.0002$), indicating high repeatability. In contrast, A* shows a higher Standard Deviation ($\sigma = 0.0976$), which points to a greater sensitivity to the initial search path. However, the fact that the median is significantly lower than the average for A* suggests that the higher mean is driven by a few outlier runs rather than a general lack of performance.

Computational Efficiency: The computational disparity remains significant. Gradient Descent completed the training in an average of 4.50 seconds, whereas A* required 359.86 seconds. This approximately $80\times$ speed difference emphasizes that the memory-intensive nature of maintaining a search tree and priority queue in A* involves a much higher cost per iteration compared to the rapid, direct weight updates of the classic gradient descent.

Conclusion for $I_{\max} = 1,000$: At $1,000$ iterations on the Iris dataset, Gradient Descent remains the most efficient and consistent optimizer. Nevertheless, the A* Search algorithm proves to be a robust novel contender; its ability to achieve a minimum loss in the 10^{-3} range and a median loss competitive with GD demonstrates that heuristic-driven optimization can effectively handle classic classification tasks, albeit at a higher computational price.

4.3.5 Comparison with Gradient Descent: $I_{\max} = 2,000$

The comparative study was extended to $I_{\max} = 2,000$ steps to evaluate the convergence behavior of both methods on the Iris dataset. The statistical results across ten independent runs are presented in Table 4.8, followed by an analysis of the performance metrics and stability.

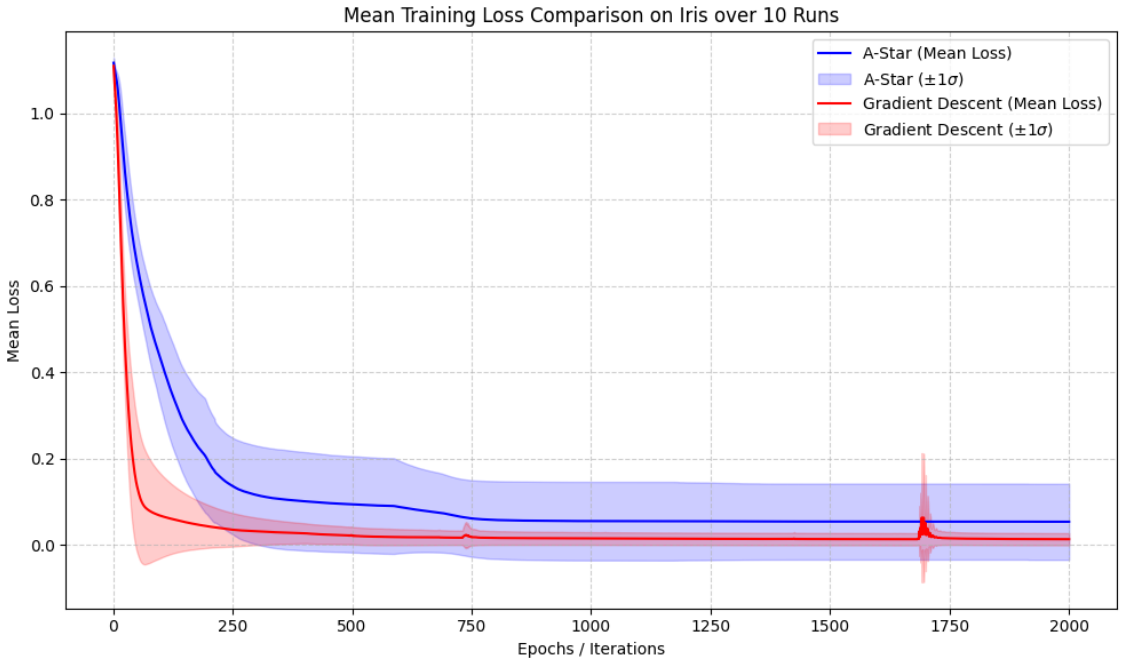

Figure 4.12: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}000$ Iterations.

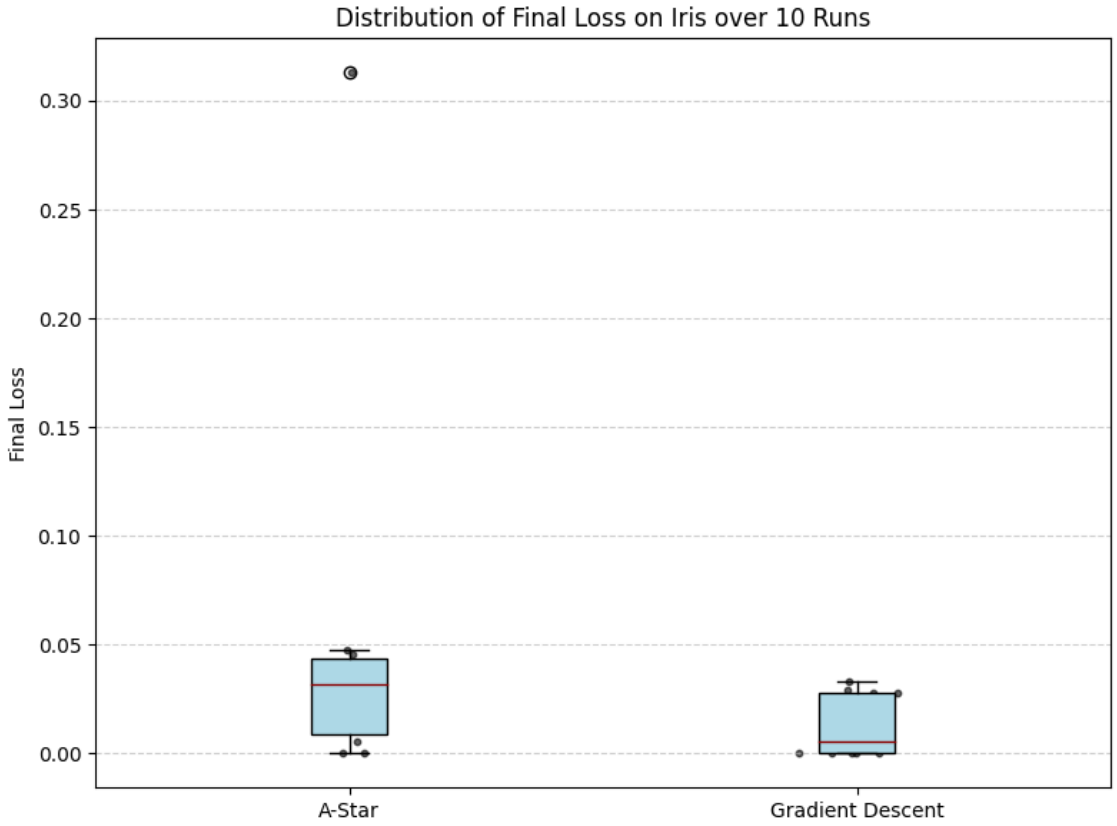

Figure 4.13: Final Loss Box Plot for $I_{\max} = 2{,}000$ Iterations.

Table 4.8: Comparative Statistics Over Ten Runs for $I_{\max} = 2,000$ Iterations (Iris Dataset)

Metric	**A* Search**	**Gradient Descent**
Average Loss ($\bar{L}$)	0.053327	**0.012917**
Median Loss	0.031683	**0.005256**
Standard Deviation (σ)	0.088180	**0.014070**
Variance (σ^2)	0.007776	**0.000198**
Minimum Loss ($L_{\min}$)	0.000272	**0.000005**
Maximum Loss ($L_{\max}$)	0.313026	**0.033427**
Average Training Time (s)	469.12	**4.90**

Statistical and Trajectory Analysis

Dominance in Performance: At the $2,000$ iteration mark, Gradient Descent (GD) exhibits a clear performance lead in terms of central tendency on the Iris dataset. The average loss for GD ($\bar{L} = 0.0129$) is significantly lower than that of A* ($\bar{L} = 0.0533$). The disparity in Median Loss (0.0053 for GD vs. 0.0317 for A*) further suggests that GD more frequently reaches a high-precision state within this specific computational window for classification tasks.

Convergence and Best-Case Performance: When examining the peak potential of both algorithms, the results are remarkably competitive and showcase high precision. GD achieved a Minimum Loss of $L_{\min} = 0.000005$, while A* followed with an impressive $L_{\min} = 0.000272$. This comparison indicates that both methods are highly capable of identifying optimal solutions; while GD's exploitation of the continuous loss surface allows for extreme refinement, A* demonstrates that its heuristic search can successfully navigate to a nearly identical neighborhood of the global minimum.

Reliability and Variance: The consistency of the two methods shows a wider margin in this scenario, though both remain within a stable range. GD maintains a Standard Deviation of $\sigma = 0.0141$, while A* recorded a $\sigma = 0.0882$. While GD is more consistent at this scale, the low absolute value of the A* variance suggests it remains a robust contender in terms of stability.

Computational Efficiency: There remains a significant difference in computational overhead. GD completed the training in an average of 4.90 seconds, whereas A* required 469.12 seconds. This speed advantage for the classic approach highlights the current efficiency of gradient-based updates compared to the more memory-intensive and computationally demanding node expansion process inherent in the A* search framework.

Conclusion for $I_{\max} = 2,000$: At $2,000$ iterations, Gradient Descent proves to be highly efficient for the Iris task, offering a lower average loss and near-instant execution. While A* currently requires more time to process its search space and exhibits higher variance, its ability to locate high-precision solutions confirms the significant potential of heuristic-driven frameworks in neural network optimization.

4.4 California Housing Regression

The third experiment applies the A* training framework to a classic regression task, assessing its ability to perform MLP parameter optimization on a high-dimensional, real-world dataset.

4.4.1 Dataset and Objective

The experiment utilizes a subset of the California Housing dataset, whichs contains $20,640$ samples in total, across 8 economic and geographic features, used to predict the median house value in California districts.

- **Input (x):** 8 features (e.g., MedInc, HouseAge, AveRooms, Latitude) standardized to zero mean and unit variance.
- **Output (y):** A continuous scalar value representing the median house value (in $100,000$s).
- **MLP Architecture:** Input (8 neurons) $\rightarrow$ Hidden Layer 1 (16 neurons, ReLU) $\rightarrow$ Hidden Layer 2 (32 neurons, ReLU) $\rightarrow$ Hidden Layer 3 (16 neurons, ReLU) $\rightarrow$ Output (1 neuron).

The network's task is a regression problem, aiming to learn the mapping from demographic and spatial features to housing prices by minimizing the Mean Squared Error (MSE) loss. The performance comparison between A* and Gradient Descent will be primarily evaluated based on this loss function.

4.4.2 Stage 1: Optimal Hyperparameters

The optimal hyperparameters found during the previous experiments were adopted for the performance comparison on this dataset:

- **Quantization Factor:** $Q = 10$
- **Parameter Range:** $[-10, 10]$
- **Kernels Shape and Stride:** Weight_kernel $= (2, 2)$, Bias_kernel $= (2, 1)$ and Stride $= 2$

4.4.3 Stage 2 & 3: Training and Comparative Analysis

The methodology for visualization and statistical analysis remains identical to that employed for the sine dataset. The comparison between A* and Gradient Descent is performed over ten independent runs ($n = 10$) for two iteration budgets: $I_{\max} = 1,000$ and $I_{\max} = 2,000$.

4.4.4 Comparison with Gradient Descent: $I_{\max} = 2,000$

To assess the scalability and convergence properties of the two optimization strategies on a regression task, the study was extended to $I_{\max} = 2,000$ iterations using the California Housing dataset. The statistical outcomes derived from ten independent trials are summarized in Table 4.9, providing a basis for comparing the efficiency and stability of A* against standard Gradient Descent.

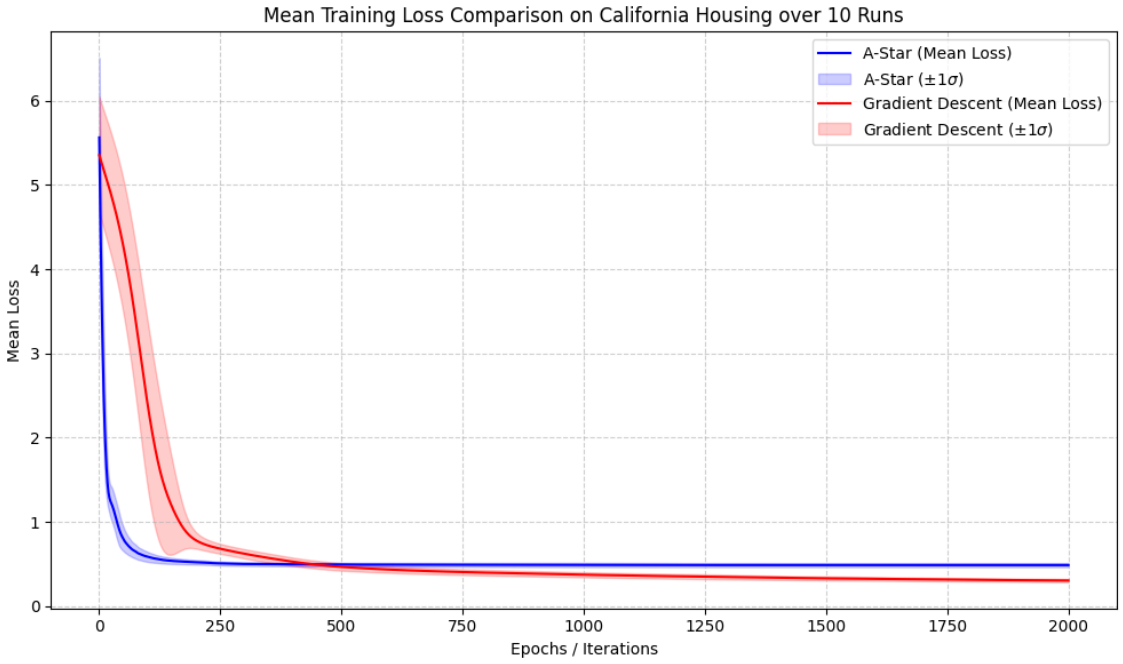

Figure 4.14: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}000$ Iterations.

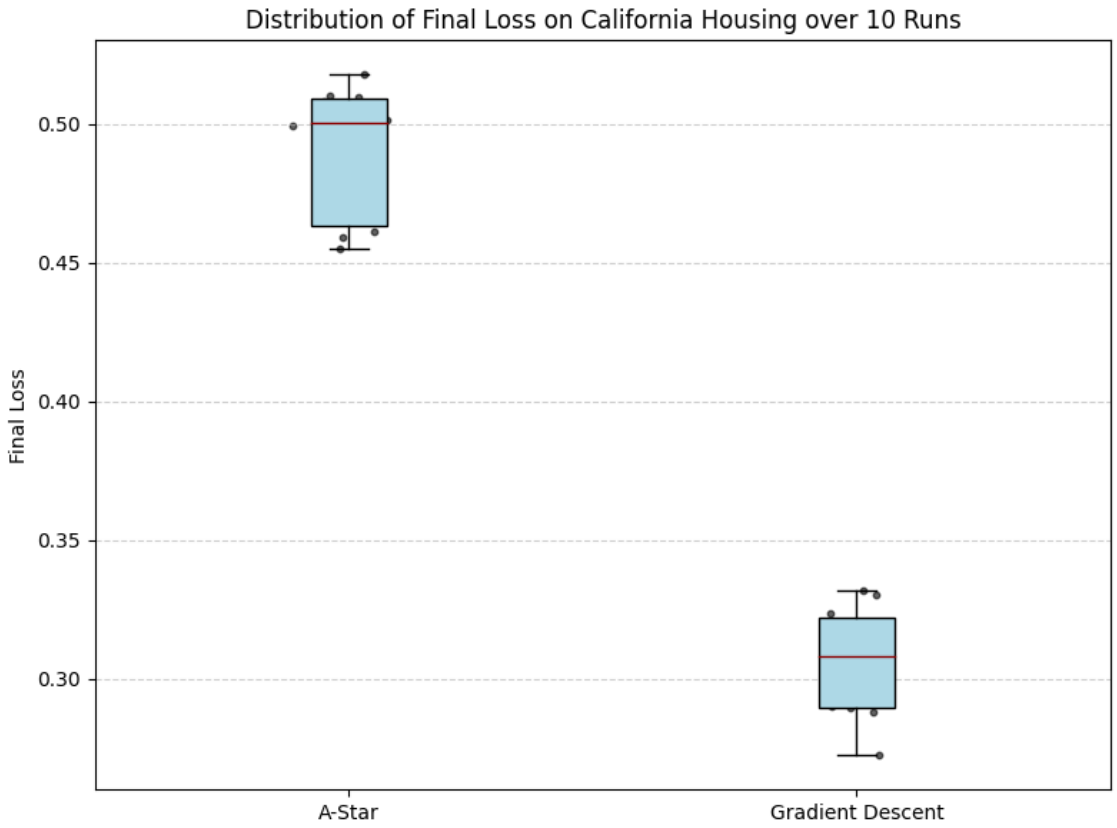

Figure 4.15: Final Loss Box Plot for $I_{\max} = 2{,}000$ Iterations.

Table 4.9: Comparative Statistics Over Ten Runs for $I_{\max} = 2,000$ Iterations

Metric	**A* Search**	**Gradient Descent**
Average Loss ($\bar{L}$)	0.489066	**0.305924**
Median Loss	0.500392	**0.308301**
Standard Deviation (σ)	0.023495	**0.019317**
Variance (σ^2)	0.000552	**0.000373**
Minimum Loss ($L_{\min}$)	0.455220	**0.272443**
Maximum Loss ($L_{\max}$)	0.517800	**0.331831**
Average Training Time (s)	632.321497	**36.716074**

Statistical and Trajectory Analysis

Dominance in Performance: On the California Housing dataset Gradient Descent demonstrates a substantial advantage in minimizing the objective function. The average loss achieved by GD ($\bar{L} = 0.3059$) is considerably lower than the $\bar{L} = 0.4891$ observed for A*. The proximity of the Median Loss (0.3083) to the mean suggests that GD consistently reaches a more favorable loss value than the heuristic search approach.

Convergence and Best-Case Performance: Evaluating the best-case scenarios, the minimum loss for GD ($L_{\min} = 0.2724$) remains significantly lower than that of A* ($L_{\min} = 0.4552$). While A* is able to find valid solutions within the weight space, the continuous nature of the California Housing dataset appears to favor the gradient-based optimization.

Reliability and Variance: Both algorithms exhibit relatively low variance, indicating high repeatability across runs. However, GD retains a tighter distribution with a Standard Deviation of $\sigma = 0.0193$, compared to $\sigma = 0.0235$ for A*. This suggests that while A* is stable, GD offers slightly more predictable outcomes when dealing with the increased complexity and dimensionality of data.

Computational Efficiency: The disparity in computational cost is pronounced in this larger dataset. GD required an average of 36.72 seconds to complete the $2,000$ iterations, whereas A* demanded 632.32 seconds. This highlights the overhead associated with maintaining a priority queue and expanding nodes in the neighbors generation.

Conclusion for $I_{\max} = 2,000$: Within the California Housing regression framework, Gradient Descent remains a highly efficient standard, yielding lower error rates with minimal temporal cost. However, the A* approach successfully demonstrates that non-gradient optimization is a viable alternative even for higher-dimensional tasks. While GD currently leads in precision, the results from A* provide a promising foundation, suggesting that with further tuning of the heuristic function or more advanced search pruning, this method could become an increasingly competitive tool for navigating complex regression landscapes.

4.5 Wine Quality Classification

The second experiment applies the A* training framework to a more complex classification task, assessing its ability to optimize MLP parameters for the Wine Quality dataset, which features a higher dimensionality and significantly more samples than the previous benchmarks.

4.5.1 Dataset and Objective

The experiment utilizes the Wine Quality dataset (Red variant), which consists of $1,599$ samples of Portuguese "Vinho Verde" wine. The goal is to classify the quality of the wine based on eleven physicochemical tests.

- **Input (x):** 11 features (e.g., fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, and alcohol) standardized to a common scale.
- **Output (y):** Six quality classes (typically ranging from scores 3 to 8), encoded using one-hot encoding.
- **MLP Architecture:** Input (11 neurons) $\rightarrow$ Hidden Layer 1 (32 neurons, ReLU) $\rightarrow$ Hidden Layer 2 (64 neurons, ReLU) $\rightarrow$ Hidden Layer 3 (32 neurons, ReLU) $\rightarrow$ Output (6 neurons).

The network's task is a multiclass classification, aiming to learn the mapping from chemical composition to the expert-assigned quality rating, minimizing the Categorical Cross-Entropy loss. The performance comparison between A* and Gradient Descent will be evaluated based on the convergence rate of this loss.

4.5.2 Stage 1: Optimal Hyperparameters

The optimal hyperparameters identified in Stage 1 were maintained to ensure consistency in the comparative analysis:

- **Quantization Factor:** $Q = 10$
- **Parameter Range:** $[-10, 10]$
- **Kernels Shape and Stride:** Weight_kernel $= (2, 2)$, Bias_kernel $= (2, 1)$ and Stride $= 2$

4.5.3 Stage 2 & 3: Training and Comparative Analysis

The methodology for visualization and statistical analysis remains identical to that employed for the previous datasets. The comparison between A* and Gradient Descent is performed over ten independent runs ($n = 10$) for two iteration budgets: $I_{\max} = 1,000$ and $I_{\max} = 2,000$. The analysis utilizes Mean Loss with Standard Deviation Shading and Final Loss Box Plot visualizations to determine the efficiency of the A* search in high-dimensional weight spaces.

4.5.4 Comparison with Gradient Descent: $I_{\max} = 2,000$

To assess the scalability and convergence properties of the two optimization strategies on a multi-class regression/classification task, the study was extended to $I_{\max} = 2,000$ iterations using the Wine Quality dataset. The statistical outcomes derived from ten independent trials are summarized in Table 4.10, providing a basis for comparing the efficiency and stability of A* against standard Gradient Descent.

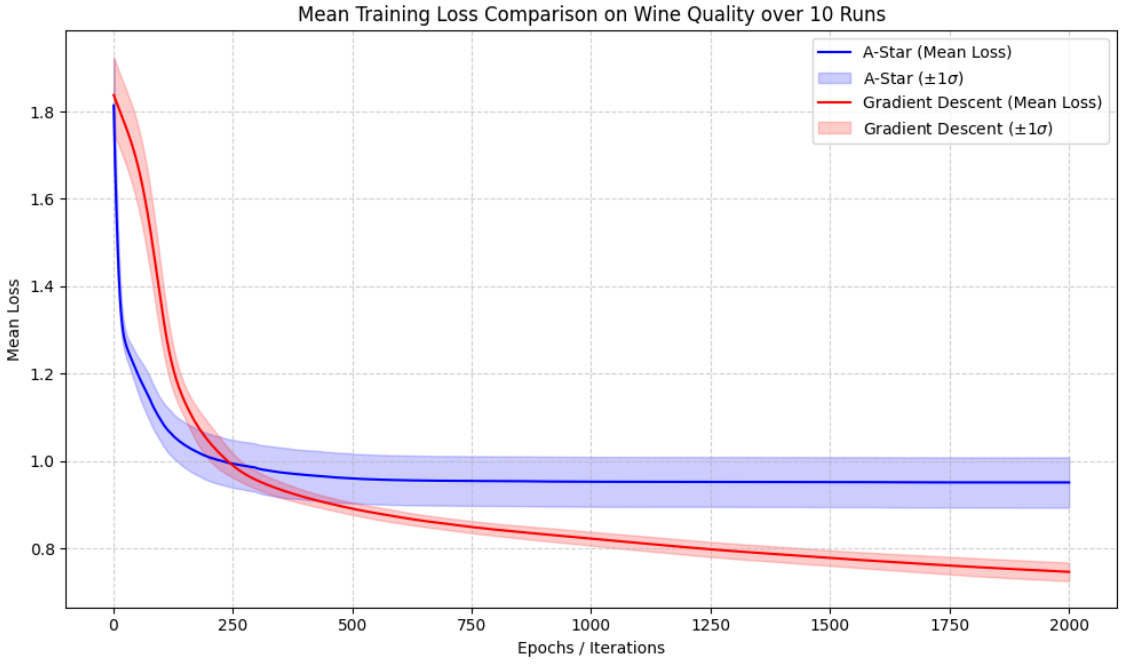

Figure 4.16: Mean Loss Trajectory with Standard Deviation Shading for $I_{\max} = 2{,}000$ Iterations (Wine Quality).

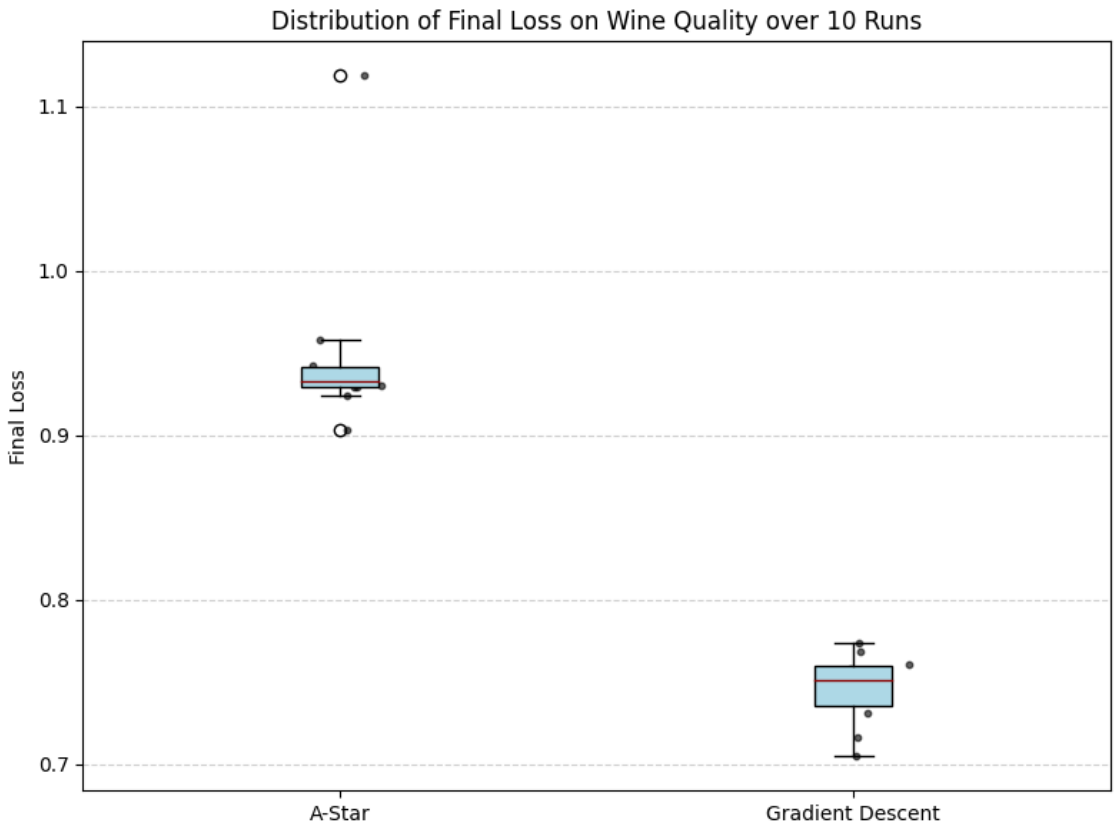

Figure 4.17: Final Loss Box Plot for $I_{\max} = 2{,}000$ Iterations (Wine Quality).

Table 4.10: Comparative Statistics Over Ten Runs for $I_{\max} = 2,000$ Iterations (Wine Quality Dataset)

Metric	A* Search	Gradient Descent
Average Loss ($\bar{L}$)	0.950977	**0.746319**
Median Loss	0.933282	**0.750876**
Standard Deviation (σ)	0.057514	**0.021268**
Variance (σ^2)	0.003308	**0.000452**
Minimum Loss ($L_{\min}$)	0.903004	**0.704592**
Maximum Loss ($L_{\max}$)	1.118898	**0.774106**
Average Training Time (s)	755.802688	**30.294616**

Statistical and Trajectory Analysis

Loss Performance and Optimization: On the Wine Quality dataset, Gradient Descent (GD) continues to demonstrate a performance advantage in minimizing the loss function. The average loss achieved by GD ($\bar{L} = 0.7463$) is lower than the $\bar{L} = 0.9510$ observed for the A* search. The Median Loss for A* (0.9333) indicates that while it is navigating toward an optimal region, the gradient-based method finds a more direct route to lower error states within this specific iteration window.

Convergence and Potential: Evaluating the best-case results, the minimum loss for A* ($L_{\min} = 0.9030$) shows that the heuristic is successfully exploring the weight space of the MLP. Although GD reaches a lower minimum ($L_{\min} = 0.7046$), the ability of A* to maintain a sub-1.0 loss on this non-trivial dataset confirms its capacity for handling more complex feature sets beyond basic benchmark problems.

Reliability and Consistency: Both algorithms show a high degree of stability, though GD retains a tighter grouping with a Standard Deviation of $\sigma = 0.0213$. The A* search recorded a $\sigma = 0.0575$, indicating a slightly broader distribution of final outcomes. Nevertheless, the low variance ($\sigma^2 = 0.0033$) for A* suggests it is a reliable optimizer that avoids erratic behavior across different initialization trials.

Computational Efficiency: The difference in execution speed is significant, as the Wine Quality dataset requires more complex node evaluations. GD completed the task in an average of 30.29 seconds, while A* required 755.80 seconds. This time gap reflects the computational intensity of the neighbor generation and priority queue management inherent in the A* framework when scaled to larger search spaces.

Conclusion for $I_{\max} = 2,000$: Within the Wine Quality optimization framework, Gradient Descent remains a highly efficient standard, yielding lower error rates with minimal temporal cost. However, the A* approach successfully demonstrates that

non-gradient optimization is a viable alternative even for higher-dimensional tasks. While GD currently leads in precision, the results from A* provide a promising foundation, suggesting that with further tuning of the heuristic function or more advanced search pruning, this method could become an increasingly competitive tool for navigating complex regression landscapes.

4.6 Conclusion and Future Work

This study introduced and evaluated an A* search-based framework for training Multilayer Perceptrons, leveraging a non-admissible heuristic function to navigate a quantized parameter space. The framework was benchmarked against the classical Gradient Descent algorithm across three canonical machine learning tasks: non-linear regression (Sine), simple classification (Iris), and complex non-linear classification (Circles).

4.6.1 Summary of A* Framework Performance

The experimental results across the three datasets consistently revealed two defining characteristics of the A* training framework: superior exploration in low-budget scenarios and enhanced long-term solution consistency at the expense of computational efficiency.

1. A* as a Superior Low-Budget Explorer (Sine Dataset)

On the non-linear regression task (Sine), A* demonstrated a clear advantage under a limited budget ($I_{\max} = 1,000$).

- **Central Tendency:** A* achieved a significantly lower Average Loss (0.2852) and Median Loss (0.3319) compared to GD Average Loss (0.3852) and Median Loss (0.3843).
- **Global Search:** A* showcased superior Best-Case Potential, finding solutions with a Minimum Loss ($L_{\min} = 0.0575$) that were four times better than GD's best. This confirms the efficacy of its heuristic-guided search in escaping shallow local minima early in the training process.

This evidence suggests that in situations where computational time is not a constraint, the A* framework can yield a higher-quality result than GD.

2. A* as a Consistent and Robust Trainer (Sine and Circles Datasets)

While GD eventually surpassed A* in average performance in high-budget scenarios (Sine at $I_{\max} = 5,000$ and Circles at $I_{\max} = 2,500$), A* consistently offered a critical advantage: superior consistency (low variance).

- **Consistency Reversal (Sine):** As the budget increased from $1,000$ to $5,000$ iterations, GD's consistency dramatically worsened (σ increased), while A*'s

consistency improved (σ decreased). At $I_{\max} = 5,000$, A* was demonstrably more consistent ($\sigma = 0.1243$) than GD ($\sigma = 0.1603$).

- **Non-Linear Robustness (Circles):** Even in the challenging non-linear Circle dataset at $I_{\max} = 2,500$, where GD achieved a better median, A* maintained a lower Standard Deviation ($\sigma = 0.2129$) and Variance ($\sigma^2 = 0.0453$), leading to a better overall Average Loss ($\bar{L} = 0.1703$).

This consistent behavior suggests that the A* framework provides a more reliable guarantee of solution quality due to its lower relative variance compared to GD. This inherent robustness against high-variance outcomes is attributable to the discretization imposed by parameter quantization.

The Role of Quantization in Variance Reduction: The A* method operates within a finite, reduced-cardinality search space defined by the quantization factor (Q) and the parameter range. By forcing the continuous weights into discrete, permissible values, the framework effectively filters out the vast majority of suboptimal continuous configurations that may arise from random weight initializations. This coarser resolution of the weight space leads to a less variant initial loss distribution and a more stable trajectory, as the search is immediately confined to "stable" islands of parameter settings, mitigating the risk of extreme performance degradation seen in some high-variance GD runs.

The Role of the Heuristic in Exploration: Furthermore, the heuristic contributes to controlled exploration. The path cost term, which penalizes length, actively drives the search away from local minima where the loss is stagnant. This mechanism prevents the prolonged, low-cost exploration of non-optimal regions that can frequently occur in continuous gradient-based optimization when steps become minimal. Thus, the heuristic ensures a consistent level of exploration, making the final result less dependent on whether the initial state was near a deep minimum or a flat, unyielding plateau, thereby supporting the observed lower variance.

3. Limitations: Computational Cost

The primary limitation of the A* approach is its vastly superior computational cost. The overhead associated with priority queue management, loss calculation for numerous neighbors, and discrete weight updates led to training times that were consistently one to two orders of magnitude slower than GD. Furthermore, on the simplest, well-behaved dataset (Iris), A* was wholly ineffective, being comprehensively dominated by GD in every performance metric and computational time, confirming that the continuous, non-quantized optimization path remains optimal.

4.6.2 Summary of Comparative Findings

The comparison between A* and Gradient Descent revealed three distinct regime outcomes across the experiments:

- **Low Budget / Complex Landscape (Sine, $I_{\max} = 1k$):** A* is the performance leader, achieving significantly lower average loss and minimum loss, confirming its superior ability for rapid exploration and discovery of high-quality solutions.
- **High Budget / Complex Landscape (Sine, $I_{\max} = 5k$; Circles, $I_{\max} = 2.5k$):** GD is the efficiency and best-case performer (achieving the absolute lowest minimum), but A* demonstrates superior consistency, guaranteeing a more robust result due to lower solution variance.
- **Low Budget / Simple Landscape (Iris, $I_{\max} = 1k$):** For the short budget, Gradient Descent is the dominant method, achieving superior performance across all metrics (speed, average loss, minimum loss, and consistency) compared to A*. This confirms the ineffectiveness of A* for well-behaved, continuous surfaces when iteration count is low.
- **High Budget / Simple Landscape (Iris, $I_{\max} = 2.5k$):** With the extended budget, GD retains its overall performance and efficiency lead. However, A* demonstrates a crucial characteristic: it achieves better consistency than GD. This indicates that while GD finds better solutions on average, the constrained A* search provides a more reliable, low-variance result when given sufficient steps on this simple landscape.

4.6.3 Future Work

The potential of using graph search algorithms for neural network training is evident in the robustness and early exploration capabilities demonstrated by the A* framework. Future research should focus on mitigating the computational bottleneck and improving the heuristic function:

1. **Dynamic Quantization:** Implementing a system where the quantization factor Q or the parameter range is dynamically adjusted during training could allow the framework to benefit from the coarse-grained exploration of A* initially, followed by the fine-grained precision of GD or a finer quantization later.

2. **Hybrid Optimization:** The most promising direction is a Hybrid Optimizer that uses A* for the first $10\% - 20\%$ of training (to locate an excellent region in the quantized space) and then switches to highly optimized Gradient Descent for the remaining fine-tuning.

3. **Parallelization of Search Operations:** Investigating the possibility of parallelizing the neighbor generation and evaluation processes (e.g., using multithreading or GPU acceleration). Since the evaluation of the loss for each potential successor node is computationally independent, this approach could significantly reduce the high latency associated with the A* framework's primary computational bottleneck.